AI LOCAL NEWS SUMMARIZATION AND ALERT BOT ON TELEGRAM

VINCENT LEE CHENG HONG

Report submitted to fulfill the partial requirements for
Bachelor of Computer Science (Hons)

SCHOOL OF COMPUTING AND CREATIVE MEDIA
UNIVERSITY OF TECHNOLOGY SARAWAK

2026

UNIVERSITY OF TECHNOLOGY SARAWAK

DECLARATION OF THESIS / UNDERGRADUATE PROJECT REPORT AND COPYRIGHT

NAME: **VINCENT LEE CHENG HONG**
ID NO.: **BCS24020018**
TITLE : AI Local News Summarization and Alert Bot on Telegram

ACADEMIC SESSION: **SEM/FEB 2025/2026**

I declare that this thesis / project report is classified as:

	CONFIDENTIAL	(Contains confidential information under Official Secret Act 1972)*
	RESTRICTED	(Contains restricted information as specified by the organization where research was done)*
√	**OPEN ACCESS**	I agree that my thesis to be published as online open access (full text)

I acknowledged that University of Technology Sarawak reserves the right as follows:

1. The thesis / project report is the property of University of Technology Sarawak.
2. The Library of University of Technology Sarawak has the right to make copies for the purpose of research only.



(VINCENT LEE CHENG HONG)

Date: 25/5/2026

Certified by:



(MS. KHAIRUNNISA IBRAHIM)

Date: 25/5/2026

Note: If the thesis / project report is CONFIDENTIAL or RESTRICTED, please attach with the letter from the organization with period and reasons for confidentiality or restrictions

DECLARATION

I hereby declare that this report is the result of my original work except for citations and quotations which have been duly knowledge. I also declare that it has not been previously and concurrently submitted for candidature of any other degree in University of Technology Sarawak or other institution.

Signature : 

Name : VINCENT LEE CHENG HONG

ID. No : BCS24020018

Date : 25/5/2026

APPROVAL FOR SUBMISSION

I certify that this project report entitled "**AI Local News Summarization and Alert Bot on Telegram**" has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelor of Computer Science (Hons).

Approved by,

Signature : 

Supervisor : MS. KHAIRUNNISA IBRAHIM

Date : 25/5/2026

The copyright of this report belongs to the author under the terms of copyright Act 1987 as qualified by Intellectual Property Policy of University of Technology Sarawak. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

© 2026, VINCENT LEE CHENG HONG, All rights reserved.

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my gratitude to all of those who have supported me along the way.

First and foremost, I would like to thank my supervisor, Ms. Khairunnisa Ibrahim, for her continuous guidance, shared knowledge, and invaluable advice throughout the research and development of this AI Local News Summarization and Alert Bot project.

I also wish to extend my appreciation to the 86 respondents who generously participated in my pilot study and TAM evaluation. Your time and feedback were essential in validating my research problem and evaluating the early prototype.

Finally, I would like to thank my family and friends for their endless support, patience, and encouragement during the preparation of this thesis.

ABSTRACT

The project is proposed to address digital information overload for Sarawak residents by developing an intelligent AI Local News Summarization and Alert Bot on Telegram. The objective is to automatically aggregate fragmented local updates, generate concise 30-word summaries using Small Language Models (SLMs), and deliver personalized notifications directly to users. Developed using the Waterfall methodology, the system's core architecture features a Python-based data extractor with a two-layered deduplication strategy, a lightweight Retrieval-Augmented Generation (RAG) summarization module, and a cloud-hosted PostgreSQL database (Supabase). A preliminary pilot study of 86 respondents successfully validated the problem statement, revealing that 69.8% of users frequently skip full-length local news articles due to time constraints. Evaluation of the early prototype demonstrated high user acceptance, with 88.4% finding the AI-generated summaries highly efficient and readable. Despite these positive outcomes, initial development encountered infrastructure uptime constraints which necessitating a migration to a cloud-based server (Fly.io) alongside minor inaccuracies in the in-memory RAG setup and token-based deduplication. Moving forward, the project will resolve these limitations by integrating a dedicated external vector database for semantic retrieval and incorporating official social media APIs for real-time utility alerts, finishing in a full-scale evaluation with 150 respondents.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE
	DECLARATION	iii
	APPROVAL FOR SUBMISSION	iv
	ACKNOWLEDGEMENTS	vi
	ABSTRACT	vii
	TABLE OF CONTENTS	viii
	LIST OF TABLES	xi
	LIST OF FIGURES	xii
	LIST OF ABBREVIATIONS	xiii
1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Problem Statements	3
	1.3 Objectives	4
	1.4 Project Scope	5
	1.5 Project Organization	6
2	**LITERATURE REVIEWS**	**8**
	2.1 Introduction	8
	2.2 Analysis of Related Work	10
	2.3 Methodology Review	13
	2.4 Tools Review	16
	2.5 Summary	20

3	**METHODOLOGY**		**21**
	3.1	Introduction	21
	3.2	SDLC	21
	3.3	System Architecture and Flowcharts	23
	3.4	Research Design and Evaluation	28
	3.5	Summary	32
4	**PRELIMINARY RESULTS**		**33**
	4.1	Introduction	33
	4.2	Implementation of AI Local News Summarization and Alert Bot on Telegram	33
	4.3	Preliminary Findings and Data	36
	4.4	Preliminary Analysis and Discussion	40
	4.5	Challenges Faced and Solutions	41
	4.6	Summary	42
5	**CONCLUSION AND PROJECT PLANNING**		**43**
	5.1	Conclusion	43
	5.2	Objectives Completed	44
	5.3	Project Timelines	45
	5.4	Limitations	47
	5.5	Future Works	49
	REFERENCES		**51**
	APPENDICES A: FYP1 & FYP2 GANTT CHART		**55**

APPENDICES B: QUESTIONNAIRE **57**

APPENDIX C: GOOGLE FORM RESPONSES **59**

LIST OF TABLES

TABLE	TITLE	PAGE
2.1	Comparative analysis of existing systems against the proposed project	12
2.2	Comparison Table between SDLC Model	14
2.3	Comparison of Programming Languages for AI and Backend Development	16
2.4	Comparison of Backend Database Solutions	18
3.1	Overview of the Questionnaire Structure, Survey Questions, and Rationale	29
4.1	Preliminary Statistics on Perceived Usefulness of AI Summaries	39

LIST OF FIGURES

FIGURE	TITLE	PAGE
3.1	The Waterfall SDLC Model	21
3.2	High-Level System Architecture of the AI News Bot	24
3.3	System Flowchart of the Automated Summarization Process	27
4.1	Early Prototype Interface of the Telegram News Bot	34
4.2	Terminal and Database Output Demonstrating Local Llama 3.1 Summarization	35
4.3.1	Age Group from total 86 Responses	36
4.3.2	Current Employment Status from total 86 Responses	37
4.3.3	Primary Devices Use for News Access from total 86 Responses	37
4.3.4	Preliminary Respondent Agreement on News Length and Information Overload	38
4.3.5	Interaction Rate from total 86 Responses	39
5.1	FYP1 Gantt Chart	44
5.2	FYP2 Gantt Chart	45

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
GUI	Graphical User Interface
LLM	Large Language Model
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RSS	Really Simple Syndication
SD	Standard Deviation
SDLC	Software Development Life Cycle
SLM	Small Language Model
TAM	Technology Acceptance Model
ACID	Atomicity, Consistency, Isolation, and Durability
SQL	Structured Query Language

CHAPTER 1

INTRODUCTION

1.1 Background

In recent years, there has been a shift in how peoples consume information, transitioning heavily from traditional print and broadcast media toward digital and social media platforms. Audiences are increasingly utilizing social media as a primary gateway to discover and engage with news content. Studies show that social media activity significantly amplifies traditional media viewership, with a substantial portion of traffic to news sites originating from digital social referrals (Ren et al., 2022). This evolution indicates that digital devices and platforms now dominate daily information access, fundamentally transforming how readers search and digest daily events.

Despite the richness of online news, there is a growing gap in the accessibility of dynamic local information. For communities in Sarawak, local updates such as scheduled utility cuts, specific community events, and critical emergency alerts like flash floods are essential for daily life. However, mainstream digital news applications frequently prioritize global or national headlines, it creating an information overload that causes users to miss these critical local updates. Furthermore, local news is often fragmented and trapped within closed social media communities, such as Facebook groups. These platforms are typically unstructured, filled with irrelevant noise or spam, and require users to have active accounts to browse the information.

To address the inefficiencies of reading lengthy and unstructured articles, artificial intelligence (AI) has emerged as a powerful tool for natural language processing (NLP) and text summarization. Large Language Models (LLMs) have

made significant advancements in processing and summarizing large volumes of text efficiently (Zhang et al., 2025). While most state-of-the-art LLMs are cloud-based and require costly subscription application programming interfaces (API), raising concerns about data privacy, the rise of locally deployed LLMs offers a compelling alternative. Local LLMs provide competitive NLP capabilities while ensuring data privacy, eliminating third-party subscription costs, and offering cost-efficient performance for lightweight workloads like fact retrieval and text summarization (Anjos, 2025).

Simultaneously, the preferred medium for digital communication has placed heavily towards instant messaging applications. These platforms are particularly favoured by young adults and busy working professionals due to their seamless integration into everyday routines and their capacity for rapid information delivery. Research highlights the growing utility of messaging apps in everyday news and information consumption practices among younger demographics (Kümpel, 2021). Additionally, the integration of automated bots within these instant messaging platforms enhances professional and personal communication, allowing for quick, push-based delivery of relevant content directly to a user's mobile device (Sulistian et al., 2025).

Recognizing these digital consumption trends, this project aims to bridge the information gap for Sarawak residents by developing an intelligent local news summarization and alerting bot. By aggregating fragmented local news sources and utilizing a Local LLM to automatically generate concise, 30-word briefs, the proposed system will deliver personalized, scheduled, and urgent community updates directly through the Telegram messaging app. This approach resolves the pain points of information overload and inefficient text consumption while providing a private, cost-effective, and highly accessible solution for local citizens.

1.2 Problem Statements

The digital transformation of news consumption has introduced significant challenges for local residents, young adults, and busy working professionals in Sarawak. While the volume of accessible information has exponentially increased globally, users face two primary problems regarding how they consume and receive vital, state-focused community updates.

Currently, mainstream digital news platforms prioritize global and national headlines, causing hyper-local updates—such as flash flood warnings or scheduled utility cuts—to be easily missed. Consequently, Sarawakians heavily rely on social networks and instant messaging platforms like WhatsApp and Facebook to find state-focused breaking news (Jerome et al., 2023). However, when local news is found on these platforms, it is often presented in lengthy, unstructured formats or trapped within noisy, closed social media groups that require active, manual browsing. For busy professionals and young adults who rely on instant messaging for quick, everyday information delivery (Kümpel, 2021; Sulistiani et al., 2025), reading through verbose texts is highly inefficient. Existing news bots on platforms like Telegram typically only forward full-length articles or raw Really Simple Syndication (RSS) feed links. The gap here is the lack of automated, intelligent summarization; users require concise, easy-to-read briefs rather than full-text articles to efficiently extract critical local updates.

To solve the inefficiency of reading lengthy texts, AI and LLMs have proven highly effective at text summarization (Zhang et al., 2025). However, the current standard of utilizing state-of-the-art, cloud-based LLMs requires costly subscription APIs and raises significant concerns regarding data privacy and security when processing localized or sensitive information. Relying on third-party cloud APIs poses a barrier for community-driven or independent developers due to ongoing operational costs and potential data exposure. The gap lies in the underutilization of locally deployed LLMs, which offer a secure alternative by enabling developers to keep control of their data while executing lightweight natural language tasks, such as summarization, without third-party subscription fees (Anjos, 2025).

To address these gaps, there is a critical need for a system that bridges the rapid delivery of instant messaging with the analytical power of private AI. This project proposes the development of a Telegram-based bot integrated with a Local LLM such as Llama 3.1 via Ollama. By automatically aggregating fragmented local Sarawak news and generating concise, 30-word briefs, the system will push personalized and urgent notifications directly to the user's mobile device. This solution actively resolves the issue of unstructured information overload while maintaining a free, privacy-sensitive, and highly accessible infrastructure.

1.3 Objectives

The main aim of this project is to develop an intelligent, automated local news summarization bot that bridges the information gap for Sarawak residents. Specifically, the objectives of the project are listed as below:

- To design and develop an automated backend system capable of scraping and aggregating local Sarawak news from fragmented RSS feeds and community websites.

- To implement a privacy-preserving locally deployed LLM via Ollama to automatically process and summarize lengthy articles into concise, 30-word briefs.

- To evaluate the effectiveness of the system in delivering personalized, scheduled, and urgent community updates through a customized Telegram messaging interface.

1.4 Project Scope

This research defines several boundaries to maintain focus and ensure the feasibility of the project. The scope covers the following aspects:

1. Type of Project/System: A Telegram-based automated bot system utilizing a locally hosted LLM for text summarization.

2. Functions and Features:

    - Included: Automated aggregation of local news, AI-driven summarization into 30-word briefs, user personalization filtering by news category and location, and urgency push-notifications for high-priority local incidents like flash floods.

    - Excluded: The system does not feature a standalone mobile application or graphical user interface (GUI) outside of the Telegram platform. Additionally, the bot requires the host machine such as local laptop to be online to process and push messages.

3. Targeted Users: Young adults, busy working professionals, and local community residents in Sarawak who require rapid access to state-focused updates.

4. Collaboration: This is an independent project developed for academic evaluation; no formal collaboration with government disaster agencies or corporate news organizations is required.

5. Location and Timeframe: The study focuses on local news within Sarawak, Malaysia. The development and evaluation will be conducted over the course of the FYP 1 and FYP 2 semesters (Academic Year 2025/2026).

6. Tools and Technologies Used: Python will be used for backend scripting, the Telegram BotFather API for the user interface, Ollama running Llama 3.1 for offline NLP, and web scrapers/RSS feeds for data extraction.

7. Significance of the Study: Practically, the system saves users time and mitigates information overload by delivering brief, structured updates. Technologically, it demonstrates a cost-effective and privacy-sensitive NLP framework by eliminating third-party cloud API subscriptions. Furthermore, it directly supports the United Nations SDG 11: Sustainable Cities and Communities by enhancing local disaster preparedness and ensuring residents are quickly informed about safety, infrastructure, and critical community events.

1.5 Project Organization

This report is systematically structured into five main chapters to provide a comprehensive overview of the research and development progress for FYP 1:

- Chapter 1: Introduction sets the foundation of the research. It provides the background of the study, outlines the problem statements, defines the research objectives, and establishes the scope and significance of the project.

- Chapter 2: Literature Review critically examines past studies and existing literature. It reviews the shift toward digital messaging applications, the use of AI in text summarization, compares similar existing systems, and identifies the research gaps.

- Chapter 3: Methodology coherently describes the research design, system development life cycle (SDLC), and the technical frameworks and tools utilized to plan and implement the system.

- Chapter 4: Preliminary Results presents the initial findings of the project. It includes early prototype interface designs, preliminary program test-run outputs, and the overall framework design.

- Chapter 5: Project Planning summarizes the current project status. It details the project Gantt chart, explains the schedule of research, outlines challenges encountered during FYP 1, and provides directions for future work in FYP 2.

CHAPTER 2

LITERATURE REVIEWS

2.1 Introduction

This section reviews the underlying theories and core technologies that form the foundation of the proposed intelligent local news summarisation bot. It examines the ongoing shift in digital news consumption towards instant messaging, the principles of AI in text summarisation, and the comparative architectures of cloud-based versus locally deployed LLMs.

2.1.1 Digital News Consumption and Instant Messaging

The shift in news consumption toward digital platforms and social media has fundamentally transformed audience behaviour. Rather than passively consuming news through print or broadcast, modern audiences increasingly discover and engage with news through digital platforms where social media acts as a primary gateway. Recent data indicates that social media activity significantly amplifies traditional media viewership, with up to 40% of traffic to news sites originating from social media referrals. In the local context of Sarawak, research by Jerome et al. (2023) supports this global trend, demonstrating a high demand for state-focused breaking news that is increasingly accessed via smartphones and instant messaging platforms like WhatsApp.

While instant messaging applications offer substantial benefits for rapid, real-time professional and personal communication (Sulistiani et al., 2025), their use for news consumption presents distinct challenges. The unstructured nature of messaging

groups often leads to difficulties in information management and concentration disruption due to excessive notifications (Sulistiani et al., 2025). This highlights a critical technical requirement which to effectively deliver news via messaging platforms, the information must be concise and systematically managed to prevent user overload.

2.1.2 AI and LLMs

To address the challenge of information overload, AI, specifically NLP has become an essential technological solution. In recent years, LLMs such as OpenAI's GPT-4, have made significant advancements in NLP and other AI applications. As outlined by Zhang et al. (2025), the evolution of text summarisation from early statistical methods to state-of-the-art LLMs has enabled systems to process massive volumes of unstructured text and generate highly accurate, human-like summaries. By integrating these models, automated systems can efficiently extract core facts from lengthy news reports and condense them into highly readable briefs summary.

2.1.3 The Feasibility of Local LLMs versus Cloud-Based LLMs

While the summarisation capabilities of modern AI are well-established, the architectural deployment of these models presents a significant technical consideration. Most state-of-the-art LLMs are cloud-based, which raises critical concerns regarding data privacy and security, especially when processing sensitive or proprietary community data (Anjos, 2025). Furthermore, relying on commercial cloud APIs suffers ongoing operational costs (Anjos, 2025; Xu et al., 2024).

In response to these limitations, locally deployed LLMs have started to emerge, offering a promising solution by enabling developers and organisations to maintain absolute control over their data while still benefiting from advanced AI capabilities. A recent comparative performance framework by Anjos (2025) demonstrates that while cloud-based LLMs may possess extensive pre-training on massive datasets, local LLMs such as Llama 3.1 are highly competitive for specific, lightweight NLP tasks. For the purposes of automated news summarisation, local LLMs provide a cost-

effective, privacy-preserving architecture without sacrificing the necessary reasoning and reading comprehension performance required to generate accurate news briefs.

2.2 Analysis of Related Work

This section critically evaluates existing solutions and platforms used for digital news consumption. By comparing mainstream news aggregators, social media communities, standard messaging bots, and existing academic AI aggregators, a distinct technological and user-experience gap is identified to justify the proposed system.

2.2.1 Mainstream News Aggregators

Mainstream news aggregators, such as Google News and Apple News, are widely used platforms designed to pool news from various sources. While they provide users with the ability to compare multiple perspectives efficiently, they suffer from significant usability weaknesses. Users frequently experience information overload and frustration with clickbait-style headlines that distract from serious, relevant news (Kantar Media, 2016). Furthermore, aggregators require active manual customization, which poses a barrier for less tech-engaged users who struggle with the effort needed to tailor these services to their hyper-local interests.

2.2.2 Social Media and Closed Communities

For many users, particularly young adults, social media has become the primary venue for news consumption. Recent systematic reviews show that young people heavily rely on social media and instant messaging apps for "incidental news exposure", stumbling upon news rather than actively visiting traditional news websites (Røsok-Dahl et al., 2024). In the local context of Borneo, studies indicate that platforms like Facebook and WhatsApp have exceptionally high penetration rates and are the preferred tools for content sharing due to their simplicity and integration into daily life (Othman et al., 2026). However, consuming news on these platforms is heavily flawed. Facebook groups are often unstructured, noisy, and require users to actively browse through a clutter of irrelevant comments to find credible information. Despite youth reliance on these platforms, there remains a severe lack of innovation from the news industry in

adapting content delivery to reach young people efficiently within these messaging ecosystems.

2.2.3 Standard Telegram Bots and Channels

Recognizing the shift toward messaging apps, Telegram has emerged as a massive venue for news consumption due to its "one-to-many broadcasting affordances" via channels and bots (Herasimenka et al., 2023). However, Telegram lacks an algorithmic timeline to recommend or filter reliable news, meaning users are entirely dependent on what the channel administrators or programmed RSS bots manually broadcast. Consequently, standard Telegram news bots typically only forward raw, full-length articles or hyperlinks. Without intelligent summarization, the broadcasting nature of Telegram still leads to users having to read verbose texts, failing to solve the inefficiency of reading full articles on mobile devices.

2.2.4 Cloud-Based AI Aggregators vs. Small Language Models (SLMs)

To address the need for automated summarization, previous academic efforts have developed AI-driven news aggregators. For instance, Joshi (2023) built an automated aggregator using LLMs such as PEGASUS and BART to scrape and summarize news, publishing the results to a standard website GUI. While technologically effective, such systems rely on heavy, cloud-based models and traditional web interfaces. Relying on cloud APIs incurs ongoing financial costs and raises privacy concerns when processing sensitive local data.

In contrast, recent evaluations of Small Language Models (SLMs) with fewer than 4 billion parameters, such as the Llama 3 architecture prove that they offer a highly efficient alternative (Xu et al., 2024). Studies demonstrate that locally deployed SLMs can achieve news summarization quality comparable to massive 70-billion parameter cloud models. SLMs not only generate more concise summaries but also run efficiently on consumer-grade edge devices. This scientifically validates the transition away from costly cloud models toward locally deployed AI for lightweight tasks like news summarization.

2.2.5 Feature Comparison and Research Gap

Table 2.1: Comparative analysis of existing systems against the proposed project

Feature	Mainstream Aggregators (e.g., Google News)	Social Media Groups (e.g., Facebook)	Standard Telegram RSS Bots	Cloud AI Aggregator (Joshi, 2023)	Proposed System (Local AI Bot)
Platform Interface	Standalone App / Web	Social Media	Telegram	Web GUI	Telegram
Content Format	Full Articles & Links	Full Articles & User Comments	Full Articles & Links	Abstractive Summaries	AI-Generated 30-word Briefs
Summarization AI	None	None	None	Cloud-based LLM	Local LLM (SLM)
Operational Cost & Privacy	High data tracking / Ad-driven	High data tracking	Free / Moderate Privacy	High recurring Cloud API costs	Free / High Data Privacy
Local Sarawak Focus	Low (Prioritizes Global News)	High (Community driven)	Varies by feed	Varies	High (Targeted Scraping)

Based on the critical analysis above, a distinct gap exists in the current landscape. Mainstream news apps and standard Telegram bots overwhelm users with unstructured, full-length articles, while existing AI summarizers rely on expensive cloud

infrastructure and outdated website interfaces. There is a lack of an automated system that innovatively combines the rapid, incidental news delivery of instant messaging platforms with the privacy-preserving, cost-effective summarization power of locally deployed SLMs. The proposed Telegram-based bot fills this gap by completely bypassing heavy web GUIs and cloud costs, directly pushing concise 30-word local news briefs to busy professionals and young adults on their preferred messaging platform.

2.3 Methodology Review

This section outlines and justifies the methodological frameworks selected for the proposed system. It details the Software Development Life Cycle (SDLC) chosen to manage the project's execution and the theoretical evaluation framework used to assess the system's AI performance.

2.3.1 SDLC: The Waterfall Model

For the development of the intelligent local news summarisation bot, the Waterfall model has been selected as the primary SDLC. The Waterfall model is a sequential design process in which progress flows steadily downwards through distinct phases which are requirements analysis, design, development, testing, and implementation.

Although iterative models like Agile have grown in popularity, recent research emphasizes that the Waterfall model remains a foundational and widely used methodology in the software development industry due to its proven track record. It is particularly suited for projects where requirements are well understood upfront and are not likely to change significantly over the course of development. According to Alshamrani and Bahattab (2015), the Waterfall model is highly advantageous when project requirements are clear, fixed, and completely known from the beginning. Because the model is linear, it is simple to implement, provides structure for developers, and greatly minimizes planning overhead.

To further justify this selection, a comparative analysis of the Waterfall model against other SDLC methodologies which are Agile and the V-Model are presented in Table 2.2.

Table 2.2 Comparison Table between SDLC Model

Comparison Category	Waterfall Model	Agile Model	V-Model
Requirement Specifications	All requirements must be known and defined clearly upfront.	Keep changing requirements, even late in the development phase.	Requirements are defined early; changes made mid-way require updating all requirements and test documentation.
Flexibility to Change	Highly inflexible; it is very difficult to go back and fix errors once a phase is completed.	Highly adaptive and flexible in responding to changing requirements.	Highly rigid and considered the least flexible model.
Customer Involvement	Low; customers may have little opportunity to preview the system until the end.	High; relies on continuous face-to-face communication and active customer inputs.	Intermediate; requires reviews and approvals at each distinct stage.
Testing Approach	Testing is carried out late, only once the code has been fully developed.	Continuous; working software is delivered frequently (e.g., in	Testers are involved from the requirements phase; testing is planned parallel to

		weeks) for rapid testing.	development stages.
Suitable Project Profile	Projects with clear, fixed requirements where quality is more important than schedule.	Smaller projects requiring rapid delivery with highly skilled senior developers.	Larger organizations and projects where strict validation is required at each phase.

For this project, the system requirements such as integrating the Telegram API, utilizing a Python backend, and deploying the Llama 3.1 Local LLM are strictly defined. Furthermore, the Waterfall model is superior to Agile and V-model for this specific academic context. Agile methodologies are designed for projects with constantly changing requirements and rely heavily on continuous face-to-face communication and active inputs from a client. Because this is an independent academic project without a corporate client demanding continuous iterations, Agile's heavy overhead is unnecessary. Instead, the Waterfall model ensures that each development phase is completed within a specified time period and accompanied by proper documentation, perfectly aligning with the structured, two-semester project timeline (FYP 1 and FYP 2) mandated by the university.

2.3.2 Theoretical Evaluation Framework

Beyond the software development process, a theoretical framework is required to strictly evaluate the performance of the integrated Local LLM. Evaluating automated text summarisation is a complex task, as it must ensure that the AI accurately compresses information without losing context or introducing hallucinations.

To evaluate the summarisation capabilities of the chosen SLM, this project adopts the theoretical evaluation parameters established in recent LLM benchmarking studies. These studies emphasize that the effectiveness of news summarisation models

should be evaluated across four primary dimensions. First of all, relevance. It is capturing the main points of the article. Second, coherence that show logical flow and readability. Third, factual consistency that ensuring no false information is generated. And last but not least, summary length which demonstrate effective text compression (Xu et al., 2024; Zhang et al., 2024). By adopting this established academic framework, the proposed system's output which is the 30-word local news briefs will be systematically tested for quality, ensuring the bot meets the high standards required for delivering reliable community updates.

2.4 Tools Review

This section evaluates the core technical tools and frameworks selected for the development of the AI Local News Summarization bot, providing justifications for their selection over alternative options in the market.

2.4.1 Programming Language

For the core development of the system's backend and data extraction modules, Python was selected as the primary programming language. To justify this selection, a comparative analysis of Python against other popular backend languages, namely Java and Node.js, is presented in Table 2.3 based on several core development categories.

Table 2.3: Comparison of Programming Languages for AI and Backend Development

Comparison Category	Python	Java / C++	Node.js
AI & NLP Ecosystem	Dominant industry standard with extensive, native libraries for Machine Learning and LLMs like	Java has limited native support; C++ is used for low-level core libraries but	Growing support, but significantly lacks the depth of mature NLP libraries compared to Python.

	PyTorch and TensorFlow.	requires complex setup.	
Web Scraping Capabilities	Highly efficient with mature frameworks like BeautifulSoup and Scrapy.	Capable, but strictly typed and, requiring more complex structure code.	Excellent for scraping dynamic content, but less integrated with subsequent data-science pipelines.
Local LLM Integration	Seamless integration with local environments like Ollama.	Complex setup required for local AI hardware acceleration.	Moderate integration capabilities via APIs, but lacks native model-handling tools.
Development Speed	High; concise syntax allows for rapid prototyping and reduces development time.	Moderate; requires more time for setup and memory management.	High; excellent for asynchronous tasks but can become complex with heavy data processing.

Source: Adapted from Alaria & Kaloriya, 2023; Singh, 2025

As illustrated in Table 2.3, while Java and Node.js are powerful languages for general web backends, they fall short in the specific domains of AI integration and data extraction. Python is widely recognized as the industry standard and the most dominant programming language for AI and NLP applications (Singh, 2025). Furthermore, Python offers an extensive ecosystem of pre-built libraries for the efficient web scraping of local news sites and data analysis (Alaria et al., 2023). Its simple syntax and seamless integration with local LLM environments significantly reduce

development time and overhead compared to using other languages like Java or C++ that require more wordy structure code and lack native, robust AI prototyping support (Alaria et al., 2023; Singh, 2025).

2.4.2 Backend Database

To manage user personalization settings and bot interaction logs, a relational structured query language (SQL), PostgreSQL database hosted via Supabase was chosen. Table 2.4 provides a comparison between PostgreSQL, which is a relational database and popular NoSQL alternatives like MongoDB and Firebase.

Table 2.4: Comparison of Backend Database Solutions

Comparison Category	PostgreSQL (Supabase)	MongoDB (NoSQL)	Firebase (NoSQL)
Data Structure	Relational SQL; uses strict tables, rows, and predefined schemas with foreign key relationships.	Document-based; highly flexible and schema-less.	Document-based; relies on nested subcollections rather than relational tables.
Data Integrity & Consistency	Extremely high; strictly Atomicity, Consistency, Isolation, and Durability (ACID) compliant, ensuring strong data consistency and relationships.	Moderate; sacrifices some consistency for flexible scaling and unstructured data.	Moderate; requires manual enforcement of data relationships within the application code.

Query Performance	Excels at complex analytical queries, aggregations, and multi-table JOINs.	Faster for simple insertions of unstructured data but slower for complex relational queries.	Limited querying capabilities; requires client-side filtering for complex conditions.
Suitability for Project	Ideal for structured user profiles, relational subscription categories, and strict data integrity.	Better suited for massive, unstructured data dumps where schemas change constantly.	Better suited for real-time mobile chat apps rather than structured relational settings.

Source: Adapted from Imroz et al., 2023; Weng et al., 2025; Yazdankhah, 2025

As demonstrated in Table 2.4, while NoSQL databases like MongoDB and Firebase offer excellent flexibility for unstructured data, they are not the optimal choice for managing rigid user configurations or executing complex analytical queries (Urnikienė et al., 2026; Weng et al., 2025). The structured nature of user profiles, Telegram chat IDs, and specific news subscription preferences required for this bot benefits heavily from the strict schema and data integrity provided by a relational database like PostgreSQL (Imroz et al., 2023; Weng et al., 2025). Supabase was specifically selected because it offers the robustness of an open-source PostgreSQL environment which has been analytically proven to execute complex structured queries significantly faster than NoSQL alternatives (Urnikienė et al., 2026) while providing readily available cloud APIs that integrate seamlessly with the Python backend, ensuring a scalable, efficient, and low vendor lock-in data pipeline (Yazdankhah, 2025).

2.5 Summary

This chapter critically reviewed the literature surrounding digital news consumption, AI, and existing news aggregator systems. The review highlighted a distinct shift towards instant messaging for daily news access, alongside the growing efficiency of locally deployed SLMs over costly cloud-based alternatives. A comparative analysis of current platforms revealed a significant gap which existing systems either overwhelm users with unstructured, full-length articles or rely on expensive, privacy-intrusive cloud APIs. To address this, the proposed Telegram-based Local AI bot provides a novel, cost-effective solution.

Furthermore, a comparative technical evaluation justified the selection of the system's core technology stack. Python was identified as the dominant industry standard for AI integration and data extraction, while a relational PostgreSQL database via Supabase was proven superior to NoSQL alternatives for maintaining strict data integrity in user profile management. Finally, the methodological approach was justified, confirming that the Waterfall SDLC and established NLP evaluation frameworks will provide a structured and scientifically rigorous path for the system's development and testing.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter details the methodology adopted to develop and evaluate the AI Local News Summarization and Alert Bot on Telegram. The chapter is systematically divided into two main components, the system development approach and the research evaluation design. The system development section outlines the specific phases of the Waterfall SDLC utilized to construct the proposed system. Subsequently, the research design section describes the quantitative methodology, theoretical frameworks, and data collection instruments used to evaluate user acceptance and the overall effectiveness of the system.

3.2 SDLC

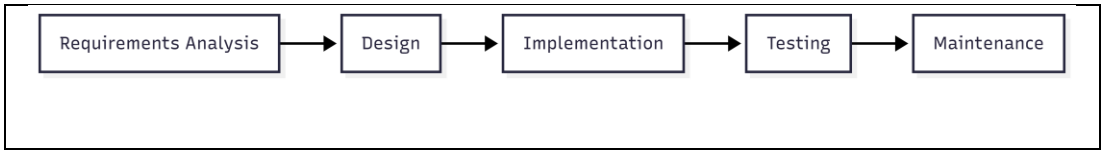


Figure 3.1: The Waterfall SDLC Model

The Waterfall model has been selected as the primary software development methodology for this project. The Waterfall model is a sequential design process in which progress flows steadily downwards through distinct phases: requirements analysis, design, implementation, testing, and maintenance (Murugaiyan, 2012;

Saravanos et al., 2023). Although iterative models like Agile are highly adaptive, the Waterfall model remains highly effective and widely used when project requirements are very well known, clear, and fixed from the beginning (Alshamrani et al., 2015). Because this project involves clearly defined requirements such as integrating the Telegram API, a Python backend, and the Llama 3.1 SLM, the Waterfall model provides the necessary structure to minimize planning overhead and ensure each phase is completed systematically within the university's academic timeline.

3.2.1 Stage 1: Requirements Analysis and Definition

In this initial phase, the specific requirements and constraints of the proposed system are gathered, analysed, and documented to establish an agreement on the software specifications. For this project, this involves defining the exact behaviour of the Telegram bot, identifying the target local Sarawak news sources to be scraped, and establishing the system constraints for deploying the local Llama 3.1 model. Proper documentation is prepared during this stage to serve as a baseline for the subsequent design process.

3.2.2 Stage 2: System and Software Design

Based on the gathered requirements, the system architecture and implementation strategy are formulated. This phase involves planning the software architecture, database conceptual schema, and overall system flow. Flowcharts are constructed to map out how the Python-based web scraper will extract news data, transmit it to the locally deployed Llama 3.1 model for 30-word summarization, and push the alerts via the Telegram Bot API.

3.2.3 Stage 3: Implementation and Unit Testing

During this phase, the system design is converted into a functional production environment through coding. The Python backend is developed to execute the automated scraping of local news feeds. Simultaneously, the Llama 3.1 model is integrated to process the scraped unstructured text and generate concise summaries. Finally, the Telegram Bot API is programmed to push these automated alerts directly

to users. Each unit of the code is tested individually to ensure it functions correctly before full system integration.

3.2.4 Stage 4: Integration and Systems Testing

Once the individual components are coded, they are integrated into a complete system and subjected to rigorous testing. This phase deals with the real-world checking of the software to ensure it meets the original requirements, such as verifying the bot's automated response time and evaluating the factual consistency of the AI-generated summaries. Any bugs, system glitches, or prompt engineering issues are identified and resolved before deployment.

3.2.5 Stage 5: Operations and Maintenance

In the final phase, the finalized AI Telegram bot is deployed and released for actual user interaction. After the software is deployed, it may require modifications, performance improvements, or error corrections based on real-world usage and user feedback to ensure continuous, uninterrupted delivery of local news alerts.

3.3 System Architecture and Flowcharts

To effectively address the problem of information overload and inefficient news consumption, the proposed system is designed to automate the extraction, summarization, and delivery of local Sarawak news. Inspired by standard automated news aggregator architectures, the system is logically divided into three primary modules: the Data Extraction Module, the AI Summarization Module, and the Delivery Interface Module.

3.3.1 High-Level System Architecture

The overall system architecture maps out the interaction between the Python backend, the locally hosted AI, and the messaging platform. Figure 3.2 illustrates this high-level architecture.

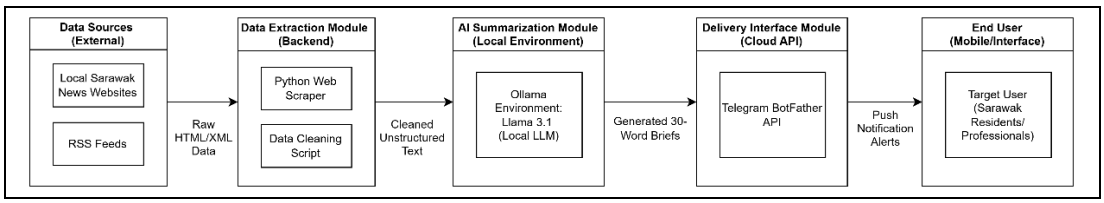


Figure 3.2: High-Level System Architecture of the AI News Bot

1. Data Extraction Module:

   This module utilizes a Python-based backend to automatically scrape and aggregate unstructured local news articles from various fragmented Sarawak-based RSS feeds and community websites. Once the raw HTML or RSS data is extracted, a data cleaning script processes the text to remove any web scraping artifacts or irrelevant noise before it is sent for analysis.

2. AI Summarization Module:

   The cleaned text is subsequently passed to the AI Summarization Module. To ensure data privacy and eliminate recurring cloud API subscription costs, this module operates entirely on a locally hosted machine using Ollama to run the Llama 3.1 LLM. The Local LLM is programmed to automatically compress the lengthy, unstructured news articles into highly concise, 30-word briefs.

3. Delivery Interface Module:

   Finally, the processed 30-word briefs are routed to the Delivery Interface Module. This module integrates the Telegram BotFather API to push the automated summaries directly to the users' mobile devices. Furthermore, this interface handles user personalization, allowing residents to configure their bot

settings to filter news by specific categories or locations, and triggers urgent push notifications for high-priority local incidents such as flash floods.

4. Two-Layered Content Deduplication Strategy:

To ensure users receive high-quality, non-repetitive news updates, the Data Extraction Module is reinforced with a robust, two-layered deduplication strategy. Because local news events are frequently covered by multiple publishers simultaneously (e.g., Borneo Post and See Hua Daily), this strategy prevents the user's digest from being flooded with redundant information.

The first layer consists of Delivery History Deduplication at the database level. When enabled, the system logs every article transmitted to each user within a specific “UserArticleDelivery” relational table. Prior to dispatching a scheduled news digest, the backend cross-references this history to guarantee that an article ID previously delivered to a specific user is permanently excluded from their future alerts.

The second layer employs Cross-Source Story Deduplication via in-memory clustering during the article ranking phase. This process begins with tokenization, which extracts meaningful alphanumeric words consisting of three or more characters while actively filtering out common stop words such as "the," "and," "breaking" (Manning et al., 2008). Following tokenization, the system utilizes Jaccard Similarity to measure the overlap of unique tokens and cluster similar articles across a multi-tier signature comparison (Manning et al., 2008). Articles are instantly grouped if they share identical recognized URLs or exact normalized titles from the same source. If exact matches are not found, the system evaluates Title Similarity against an 0.82 overlap threshold. For articles with differing titles but sufficient body content (a minimum of 18 tokens), a Body Similarity check is executed using a 0.72 threshold. Finally, the system evaluates the AI-generated summaries against a 0.58 similarity threshold. If a candidate article satisfies any of these thresholds when compared to a previously processed article, it is grouped into a unified cluster. Ultimately, only the primary article from this cluster is pushed to the user's digest, with the

various publisher sources concatenated (e.g., "Sources: Borneo Post, Sarawak Tribune") to maximize reading efficiency.

5. Lightweight Retrieval-Augmented Generation (RAG) Framework:

   To facilitate interactive user queries beyond the automated push notifications, the AI Summarization Module is augmented with a lightweight, in-memory RAG framework (Lewis et al., 2020). This configuration enables users to ask specific questions about the local news without requiring a heavy, dedicated external vector database.

   The RAG system operates by utilizing Ollama's Object Programming Interface (API) embeddings endpoint, defaulting to the nomic-embed-text model, to generate mathematical vector representations for both the user's queries and the scraped news articles. When embedding a news article into the system, the backend connect its core metadata, specifically the title, summary, category, location, and state to construct a comprehensive and context-rich text representation. To optimize processing speed and minimize redundant API calls, these generated embeddings are cached directly in the system's memory using a SHA1 hash identifier. During the retrieval phase, the backend calculates the cosine similarity between the embedded user query and a candidate pool of up to 15 recent articles (Manning et al., 2008). The system then isolates and returns the top 6 most semantically relevant results to ground the LLM's conversational responses, ensuring the bot answers user queries based strictly on the latest extracted local facts.

3.3.2 System Flowchart

The operational logic of the proposed system is sequentially mapped out in a system flowchart. The flowchart visualizes the step-by-step decision-making process of the backend system, from the initial scheduled extraction trigger to the final delivery of the message to the user. Figure 3.3 depicts the complete system flowchart.

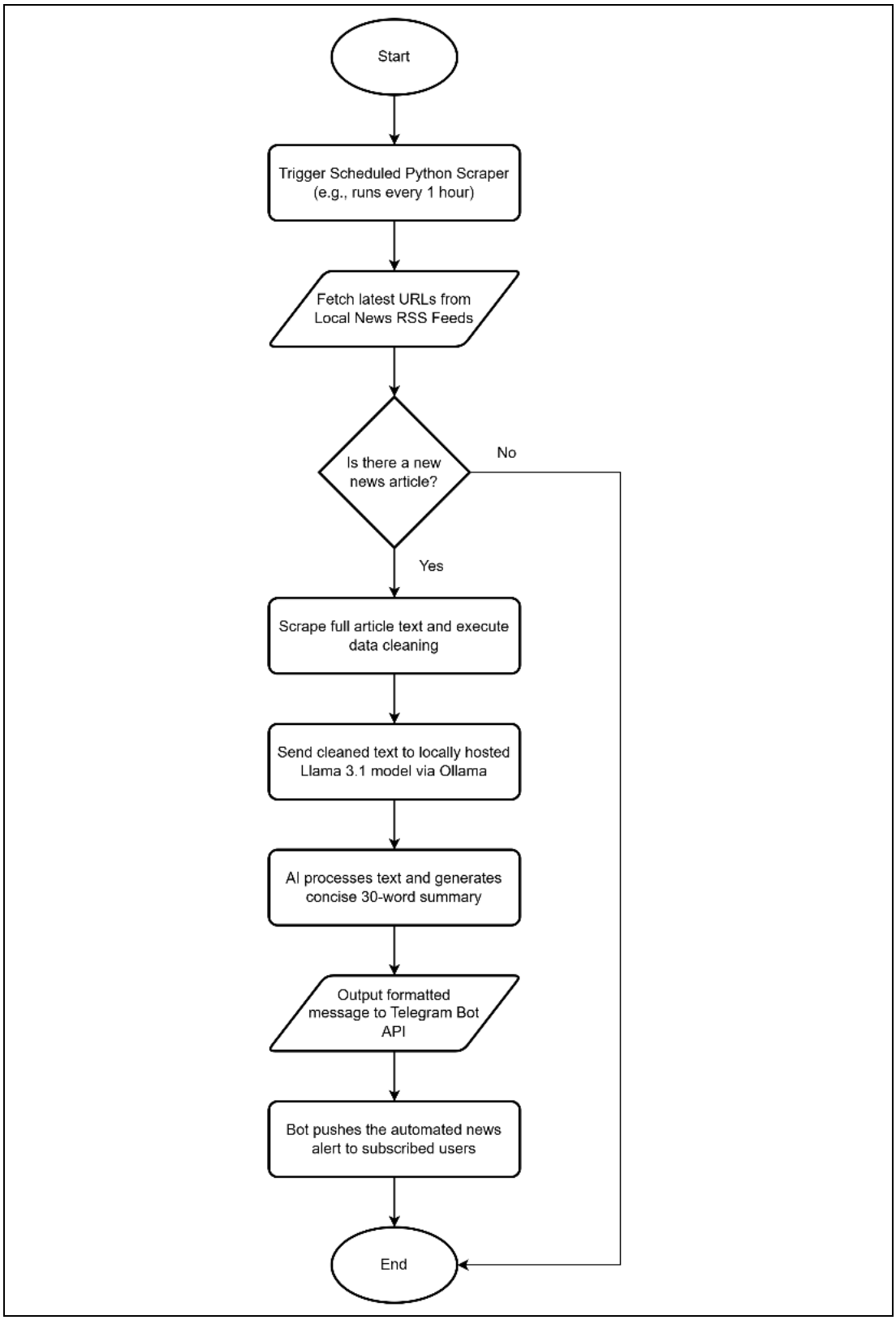


Figure 3.3: System Flowchart of the Automated Summarization Process

3.4 Research Design and Evaluation

While the Waterfall model guides the technical software development of the Telegram bot, a formal research design is required to rigorously evaluate the system's effectiveness and user acceptance once it is deployed.

3.4.1 Research Questions for Evaluation

The evaluation phase of this project is driven by specific research questions aimed at assessing the success of the developed system. The primary research question guiding this phase is: To what extent is the AI Local News Summarization Telegram bot effective in reducing information overload and delivering accurate, readable news briefs to users?

3.4.2 Research Methodology and Sampling

To answer the research question, this study adopts a Quantitative Research methodology. A quantitative approach is highly suitable for measuring user acceptance, system performance, and the perceived quality of the AI summaries through numerical data and statistical analysis. The study will utilize the Technology Acceptance Model (TAM) as the foundational theoretical framework to measure the users' behavioural intention to use the bot based on perceived usefulness and perceived ease of use.

The evaluation will be conducted in two distinct testing phases with specific sample sizes:

1. Prototype Testing (10 Testers): Initially, a smaller control group consisting of 10 testers will be recruited. This group will participate in guided experiments to test the core functionality of the Telegram bot, verify the bot's response times, and check the fundamental formatting of the AI summaries.

2. Main Quantitative Survey (150 Respondents): Following the successful prototype testing, the bot will be deployed to a larger audience. To achieve statistically valid results for the quantitative analysis, the study targets a minimum requirement of 150 respondents. The target population focuses on

local young adults, university students, and busy professionals in Sarawak who regularly consume digital news.

3.4.3 Data Collection Instrument

The primary data collection instrument for the main evaluation phase will be a structured online survey administered through Google Forms. The questionnaire will be distributed to the 150 respondents after they have interacted with the Telegram bot for a specified period. The survey will utilize a 5-point Likert scale (ranging from Strongly Disagree to Strongly Agree) to quantitatively assess user satisfaction, ease of use on the Telegram platform, and the quality of the AI-generated summaries.

To ensure a comprehensive evaluation, the questionnaire is strategically divided into four main sections, each designed to capture specific data points relevant to the project's objectives. The complete set of questions used for the TAM evaluation pilot study is attached in Appendix B for reference.

Table 3.1: Overview of the Questionnaire Structure, Survey Questions, and Rationale

Survey Section	Question Focus	Actual Survey Questions	Research Purpose / Rationale
1. Demographic and News Consumption Profile	Demographics and mobile news reliance.	• What is your age group? • What is your current employment status? • Where do you currently reside? • What devices do you primarily use to access the news?	To verify that respondents align with the target audience (young adults/professionals in Sarawak) and to establish a baseline of current mobile news reliance.

		• How often do you read or check local Sarawak news? • Which platforms do you currently use the most to get your local news? • What types of local news are most important to you?	
2. Problem Statement Validation	Information overload, length of articles, and scattered delivery.	• "I often feel overwhelmed by the amount of information and 'noise' on social media when trying to find important local news." • "I often skip reading full local news articles because they take too much time." • "Because local news is scattered across different Facebook pages and websites, I sometimes miss out on urgent community updates (like water/electricity cuts)."	To empirically validate the core problem statements defined in Chapter 1 and mathematically justify the necessity of an automated AI summarisation tool.
3. System Evaluation (TAM)	Perceived Ease of Use and Perceived Usefulness.	• Were you able to successfully access and interact with the Telegram bot?	To evaluate the deployed Telegram bot prototype based on the TAM and measure the users' overall behavioural

		• "The AI-generated news summaries were easy to read and gave me the core information quickly." • "Navigating the bot using the menu buttons (e.g., Settings, Latest, Help) was simple and straightforward." • "I found the 'Settings' feature useful for customizing the news to my specific location and interests." • "Overall, I would use this bot in my daily life to stay updated on local Sarawak news."	intention to use the system daily.
4. Open-Ended Feedback	Feature suggestions and qualitative feedback.	• Do you have any suggestions to improve the bot (More languages, etc), or did you encounter any errors? • Please leave your Telegram Handle (e.g., @yourname), email address or Phone/WhatsApp number.	To collect qualitative feedback for identifying technical limitations, gather contact details for follow-up, and guide future system enhancements for FYP 2.

3.4.4 Data Analysis Procedure

Once the data collection via Google Forms is complete, the data will be exported and prepared for quantitative evaluation. The data analysis procedure will involve computing descriptive statistics to summarize the dataset, specifically focusing on the Mean, Median, Mode, and Standard Deviation (SD) of the users' responses. The actual execution of this data analysis, the interpretation of the statistical results, and the in-depth discussion of the findings will be comprehensively presented in Chapter 4.

3.5 Summary

This chapter provided a detailed description of the methodology adopted for the project. The SDLC is guided by the Waterfall model, ensuring a structured approach through requirements analysis, system design, implementation, testing, and operations. The system architecture outlines the integration of the Python web scraper, the local Llama 3.1 model, and the Telegram API. Finally, the research design established a quantitative evaluation framework utilizing the TAM. By leveraging a Google Forms survey with 150 respondents and statistical analysis tools, the final system's effectiveness and user acceptance will be thoroughly assessed.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the preliminary outcomes and early development progress of the AI Local News Summarization and Alert Bot. Because this report represents the first phase of the project (FYP 1), the findings discussed here focus on initial system implementations rather than the final comprehensive system evaluation. This chapter is divided into three main sections: the early prototype interface and test-run outputs, the preliminary data gathered from a pilot study of 86 respondents, and the initial technical challenges encountered during the development phase. The complete system deployment and the final large-scale quantitative evaluation will be executed in the subsequent FYP 2 phase.

4.2 Implementation of AI local news summarization and alert bot on telegram

To demonstrate the feasibility of the proposed system, early prototypes of both the user interface and the backend processing modules have been developed and tested. The initial implementation focuses on establishing the connection between the Telegram platform, the AI environment, and the database.

4.2.1 System Implementation and Early Prototype

The primary user interaction point is the Telegram messaging application. The initial interface design focuses on a clean, intuitive chat layout where users can easily subscribe to specific local news alerts. Figure 4.1 illustrates the early prototype of the Telegram bot's message and the main navigation menu.

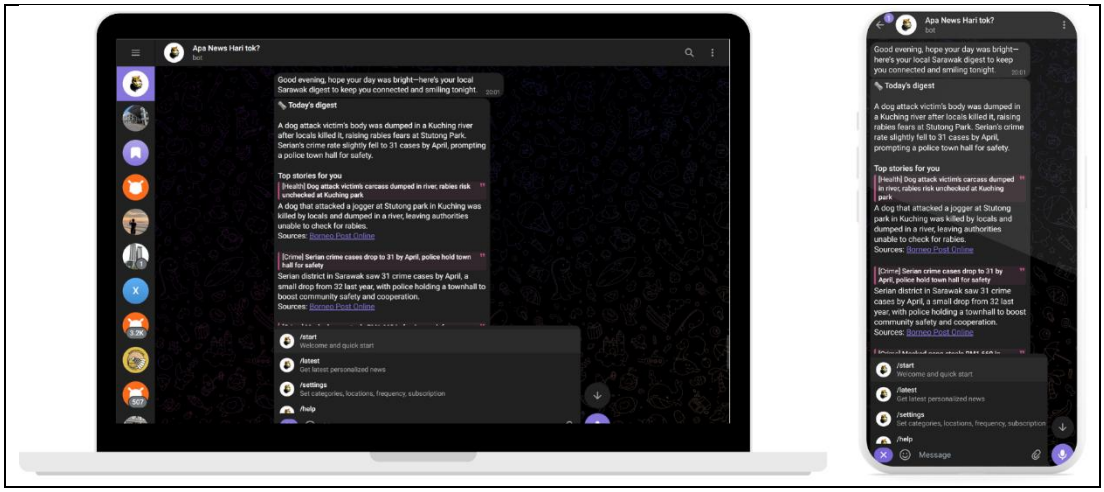


Figure 4.1: Early Prototype Interface of the Telegram News Bot

4.2.2 Preliminary Program Test-Run Outputs

On the backend, the core processing relies on the locally deployed Llama 3.1 SLM executing text summarization. Preliminary test runs have been conducted to ensure the Python RSS scraper can successfully extract text from local news sources and pass it to the Ollama environment. Figure 4.2 displays a terminal test-run output demonstrating the AI successfully compressing a lengthy news article into a concise 30-word brief.

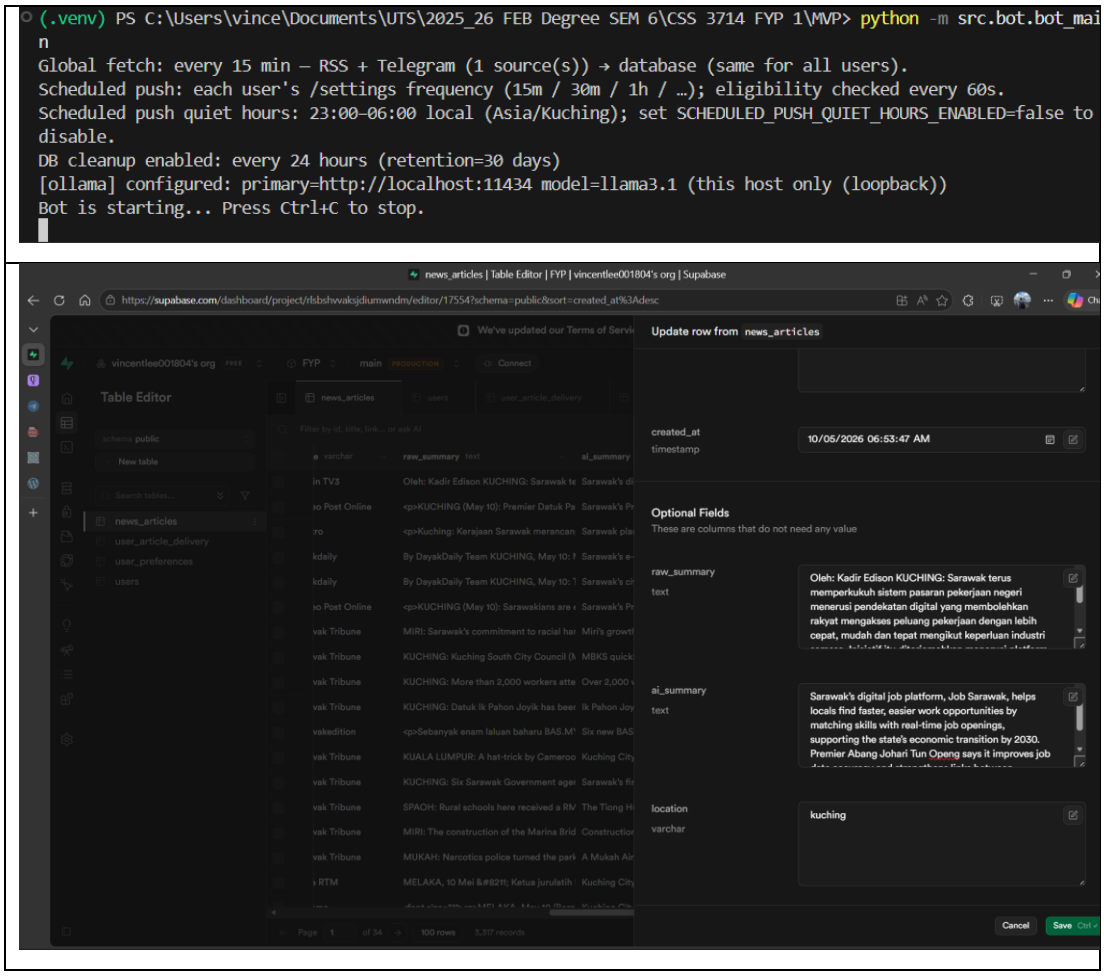


Figure 4.2: Terminal and Database Output Demonstrating Local Llama 3.1 Summarization

4.3 Preliminary Findings and Data

While the final project evaluation requires a larger sample size, a pilot study was conducted during this FYP 1 phase to gather preliminary data and validate the core problem statements. A total of 86 responses were successfully collected from the target audience.

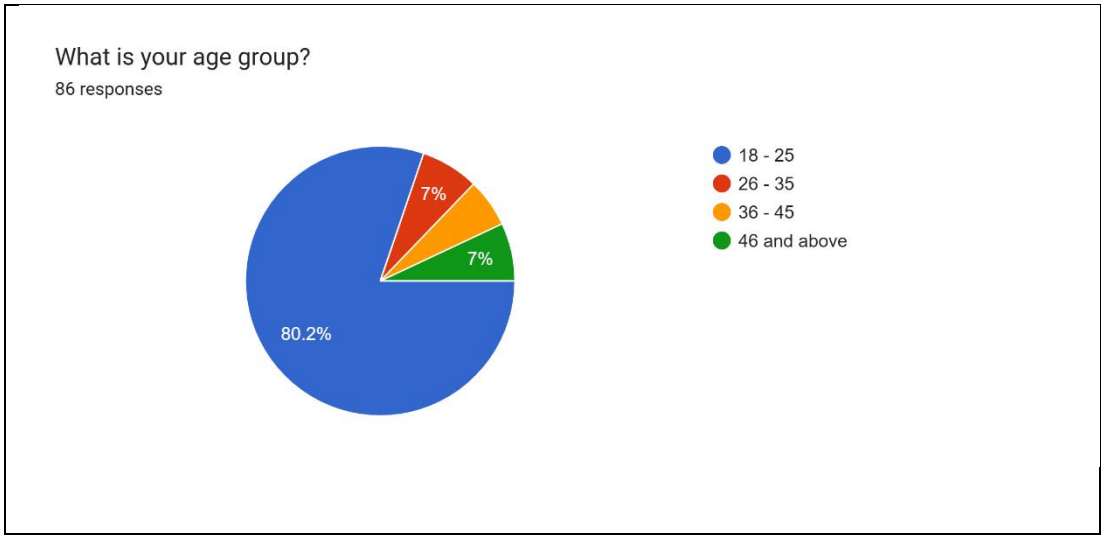


Figure 4.3.1: Age Group from total 86 Responses

The demographic breakdown aligns closely with the intended user base, with 80.2% of respondents aged between 18 and 25, and the remainder comprising young professionals and adults above 26.

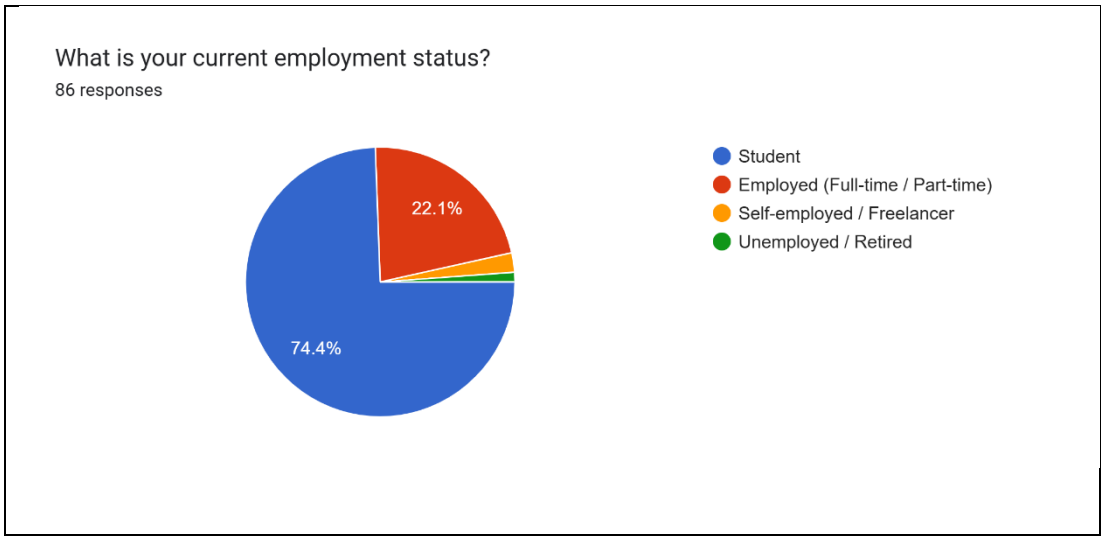


Figure 4.3.2: Current Employment Status from total 86 Responses

In terms of employment, 74.4% identified as students, and 24.4% as employed or self-employed individuals.

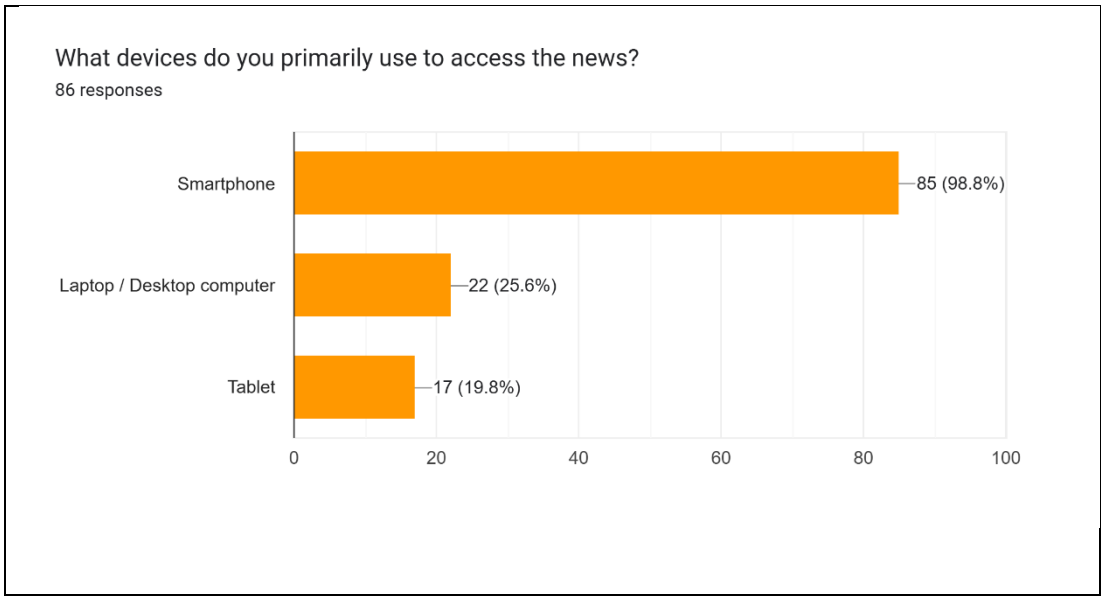


Figure 4.3.3: Primary Devices Use for News Access from total 86 Responses

Furthermore, the preliminary data confirmed high mobile device reliance, as 100% of respondents use smartphones either exclusively or combined with other devices to access daily news.

The early data strongly supports the initial problem statements regarding information overload and fragmented news delivery. As visualized in Figure 4.3.4, a significant majority of the early respondents indicated that current digital news articles are too lengthy and scattered. Specifically, 69.8% agreed or strongly agreed that they often skip reading full local news articles due to time constraints, and 73.3% reported missing out on urgent community updates because local news is scattered across different social media pages.

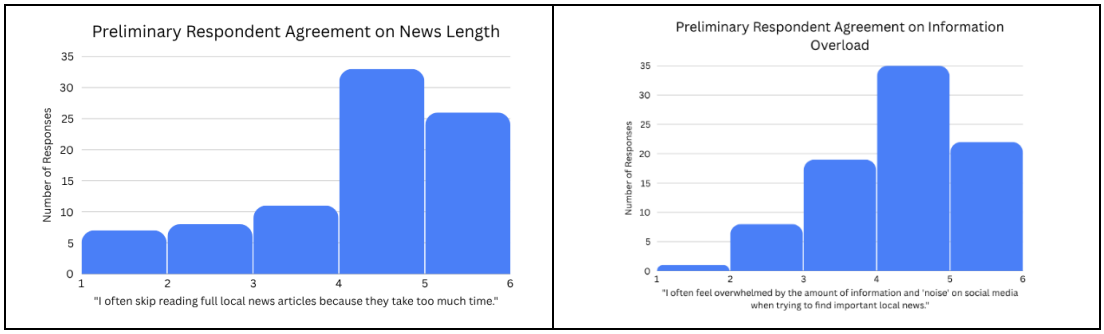


Figure 4.3.4: Preliminary Respondent Agreement on News Length and Information Overload

Following the problem identification, the survey evaluated the respondents' experiences testing the early prototype of the Telegram bot. A highly successful initial interaction rate was recorded, with 95.3% of the pilot users successfully accessing and interacting with the bot.

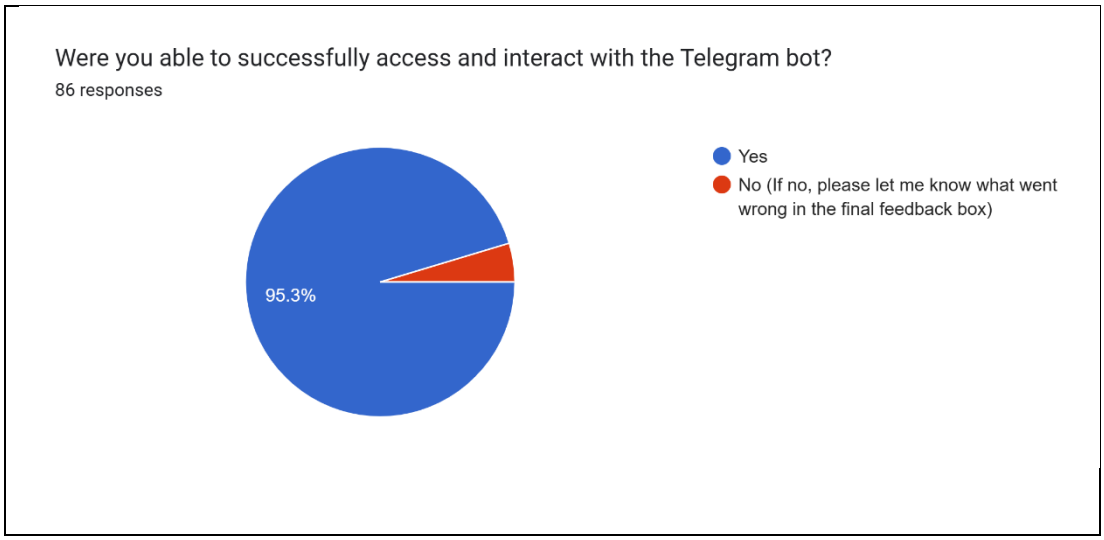


Figure 4.3.5: Interaction Rate from total 86 Responses

Table 4.1 summarizes the preliminary descriptive statistics based on the TAM, measuring the users' perceived usefulness and ease of use on a 5-point Likert scale (1 = Strongly Disagree to 5 = Strongly Agree).

Table 4.1: Preliminary Statistics on Perceived Usefulness of AI Summaries

Evaluation Criteria	Mean	Median	Mode	SD	Agreement Percentage (Rated 4 or 5)
"The AI-generated news summaries were easy to read and gave me the core information quickly."	4.22	4	5	0.92	88.4%
"I found the Settings feature useful for customizing the news to my specific location and interests."	4.24	4	5	0.86	83.7%

"Navigating the bot using the menu buttons was simple and straightforward."	4.17	5	5	1.05	77.9%
"Overall, I would use this bot in my daily life to stay updated on local Sarawak news."	4.19	4	5	0.86	82.6%

The raw data and complete Google Form responses collected from the 86 pilot respondents used for this statistical analysis are provided in Appendix C.

4.4 Preliminary Analysis and Discussion

The descriptive statistics gathered from the pilot study provide crucial empirical validation for the theoretical justifications established in Chapters 1 and 2. The high agreement percentages regarding information overload (66.3%) and scattered news sources (73.3%) confirm the core problem statement which Sarawak residents are actively seeking local news, the current unstructured delivery on platforms like Facebook is highly inefficient. Furthermore, the fact that nearly 70% of users state to skipping full-length articles mathematically justifies the necessity of deploying a NLP summarisation tool.

The evaluation of the prototype solution generated highly positive user acceptance, demonstrating the practicality of the proposed architecture. The integration of the local Llama 3.1 model successfully addressed the reading-time constraint, evidenced by the 88.4% of respondents who found the 30-word AI briefs summary easy to read and efficient for extracting core information. Additionally, the high perceived usefulness of the customization settings (Mean: 4.24) validates the decision to implement a robust, relational PostgreSQL database (Supabase) capable of managing individual user preferences and locations, as opposed to simple mass-broadcasting methods used by standard Telegram RSS bots.

Finally, open-ended feedback collected during the survey provided valuable insights for future development. While the overall behavioural intention to use the bot was high (82.6%), several respondents suggested the implementation of multi-language support such as Bahasa Melayu and Mandarin and requested a more simplified user interface for the customization settings. These empirical insights and usability concerns will directly advise the iterative improvements planned for the system's final deployment and comprehensive evaluation in FYP 2.

4.5 Challenges Faced and Solutions

During the preliminary development and testing phases, a significant infrastructure challenge was encountered regarding system uptime and continuous availability. Acknowledging and addressing this constraint was vital for refining the system prior to its final deployment and evaluation in FYP 2.

Initially, the system's architecture was entirely localized. The Python backend, the PostgreSQL database, and the Llama 3.1 model via Ollama were all hosted and executed on a single personal laptop. While this local deployment ensured maximum data privacy and zero operational cost, it proved highly impractical for the user testing phase. A personal machine cannot realistically remain powered on 24 hours a day, which meant the Telegram bot would frequently go offline. Consequently, pilot testers were unable to interact with the bot or receive scheduled news alerts reliably during their preferred times.

To resolve this uptime limitation and ensure continuous availability during the testing period, the system architecture was migrated to a strong cloud-based infrastructure. The Python backend was successfully deployed to a Fly.io server, allowing the web scraper and Telegram listener to run continuously in the background. Furthermore, the localized database was upgraded to a cloud-hosted PostgreSQL database via Supabase. As justified in Chapter 2, Supabase provides superior relational data integrity and integrates seamlessly with Python, making it an ideal, scalable solution for storing user preferences and chat logs. Finally, the AI summarisation workload was transitioned from the local Llama 3.1 model to a cloud-hosted SLM from the Qwen/Ministral family. Benchmarking studies indicate that advanced SLMs

like Qwen perform comparably to Llama 3 in text summarisation tasks (Xu et al., 2024), ensuring the bot maintains high-quality, 30-word outputs while guaranteeing 24/7 reliability for the users.

4.6 Summary

This chapter outlined the preliminary progress of the AI Local News Summarization bot. The early system implementation successfully demonstrated the basic Telegram interface and verified the backend summarization capabilities of the local Llama 3.1 model. Additionally, the preliminary data collected from 86 pilot respondents successfully validated the core problem statement, confirming that users experience information overload and are highly receptive to the proposed 30-word AI briefs. Addressing the prompt engineering challenges and expanding the survey sample size will be the primary focus moving into the FYP 2 development phase.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Conclusion

This report marks the conclusion of the first phase of the Final Year Project (FYP 1). Over the development of this semester, the foundational work for the AI Local News Summarization and Alert Bot was successfully established. This included a comprehensive literature review identifying the critical gap in localized, bite-sized news delivery, as well as the formulation of a structured methodology based on the Waterfall SDLC. Furthermore, the preliminary development phase successfully produced an early prototype of the Telegram bot and validated the backend integration with cloud-based PostgreSQL (Supabase) and SLMs.

5.2 Objectives Completed

During this FYP 1 phase, the initial project objectives were successfully met:

- Objective 1, which aimed to design and develop an automated backend system capable of scraping and aggregating local Sarawak news from fragmented sources, was successfully achieved by designing the system's core architecture and ensuring a seamless data pipeline between the Python scraper, the database, and the Telegram API.

- Objective 2, focused on implementing a privacy-preserving locally deployed LLM to process and summarize lengthy articles, was successfully fulfilled during the early prototyping phase, demonstrating the system's ability to automatically generate the proposed concise, 30-word automated news briefs.

- Objective 3, which aimed to evaluate the effectiveness of the system in delivering updates through a customized Telegram interface, was successfully initiated through a pilot study involving 86 target respondents. The empirical data collected conclusively validated the core problem statement, demonstrating that users experience significant information overload and are highly receptive to the automated summaries.

5.3 Project Timelines

Effective project management is critical to ensuring the systematic completion of the remaining development phases within the academic timeframe. A detailed project timeline has been formulated to guide the transition from FYP 1 into FYP 2. This schedule outlines the specific milestones, including the completion of the backend cloud migration, full-scale user testing, and final documentation. Figure 5.1 illustrates the FYP1 Gantt chart, while Figure 5.2 show the FYP2 Gantt chart, which both serves as the primary project planning and tracking instrument for both semesters.

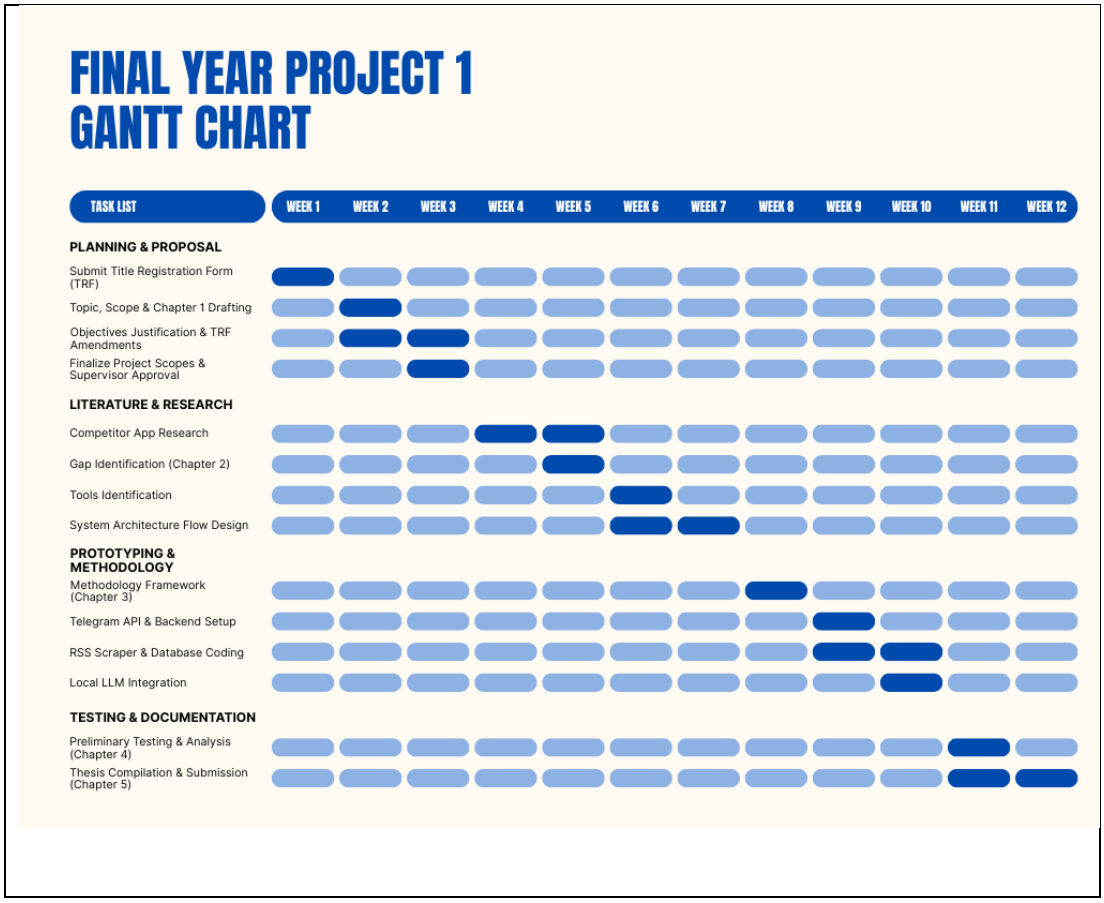


Figure 5.1 FYP1 Gantt Chart

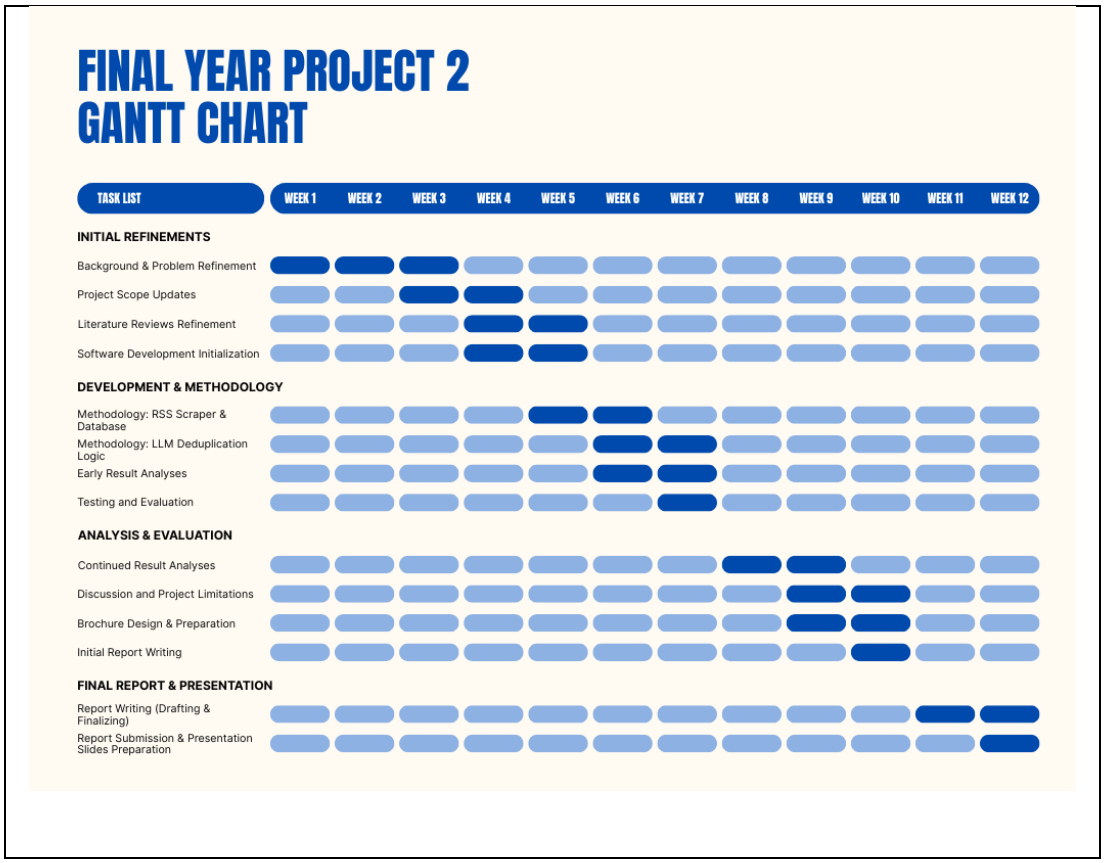


Figure 5.2 FYP2 Gantt Chart

5.4 Limitations

This section addresses the key constraints and boundaries encountered during the FYP 1 development and testing phases, which impacted the scope and generalizability of the initial findings.

5.4.1 Scale of User Validation (Sample Size)

The preliminary evaluation process was constrained by the available user pool within the first FYP 1 semester's timeframe. While functional testing and user feedback were successfully gathered, the validation was limited to a pilot convenience sample of 86 respondents. Although the results were highly positive, this small sample size limits the statistical generalizability of the perceived usefulness and ease-of-use findings to the entire target population of Sarawak.

5.4.2 Technical and Infrastructure Boundaries

A significant limitation relates to the deployment environment of the Local LLM. The initial scope intended for the system, including the Llama 3.1 model and PostgreSQL database, to be hosted entirely on a local personal machine to maximize privacy. However, a technical boundary was reached regarding system uptime; a personal laptop could not realistically remain powered on 24 hours a day to continuously scrape news and push alerts. Consequently, the system had to rely on a temporary cloud-hosted workaround via Fly.io and Supabase to ensure the bot remained accessible for pilot testers.

5.4.3 RAG Constraints

Another technical limitation pertains to the system's conversational and question-answering capabilities. Although a lightweight RAG framework was implemented to allow users to query the news, the current configuration struggles to answer user questions effectively. The existing RAG setup operates as an in-memory system that utilizes Ollama's embeddings endpoint which defaulting to the nomic-embed-text model to generate vectors. To create these text representations, the system connect the article’s title, summary, category, location, and state, caching the embeddings using a

SHA1 hash to minimize redundant API calls. During retrieval, the system calculates the cosine similarity between the embedded query and a limited candidate pool of up to 15 articles, returning the top 6 most relevant results to ground the LLM's responses. While this approach avoids the overhead of a dedicated external vector database, the simplistic connection and restricted candidate pool limit the semantic depth of the retrieval process. As a result, the system frequently fails to capture the necessary context to answer complex user questions accurately.

5.4.4 Content Deduplication Inconsistencies

During the pilot testing phase, an operational limitation was also observed regarding the system's two-layered content deduplication strategy. Although the backend employs both database-level delivery history tracking and in-memory clustering to prevent redundant news, users occasionally still received repeated news stories. This issue primarily arises from the limitations of relying on tokenization and Jaccard Similarity. When different local publishers cover the exact same event but utilize different vocabulary, sentence structures, or phrasing, the token overlap can easily fall below the system's predefined clustering thresholds. For example, the 0.82 title similarity or 0.72 body similarity. Therefore, the system fails to recognize them as the same story and pushes them as separate news, which partially weakens the bot's primary objective of reducing user information overload.

5.4.5 Data Source Extraction Constraints

A specific operational limitation was encountered regarding the extraction of critical community updates, specifically scheduled or sudden utility disruptions such as electricity and water cuts. While official utility providers in Sarawak such as Sarawak Energy Berhad and Sarawak Water Department do broadcast real-time disruption notices on their official social media channels including X (formerly Twitter), Facebook, and Telegram, extracting this data presents a distinct technical barrier. The current Data Extraction Module was designed primarily for standard HTML parsing and RSS feed extraction from news websites. Pulling automated, real-time data directly from these closed social media ecosystems requires the integration, authentication, and maintenance of platform-specific APIs, such as the Facebook

Graph API or the X API. Due to the timeline and scope constraints of the FYP1 semester, implementing these complex third-party APIs was not feasible. Therefore, the bot is currently limited to scraping local news portals for utility updates, which can delay the delivery of primary-source disruption alerts.

5.4.6 Scope and Feature Constraints

Finally, due to the timeline constraints of the FYP1 semester, complex prompt engineering for the AI model could not be fully perfected. The SLM infrequently struggled to strictly adhere to the 30-word summarisation limit, resulting in slight inconsistencies in the length of the news briefs delivered to users. Addressing these prompt constraints, integrating a more robust external vector database for the RAG framework, refining the deduplication algorithms, and finalizing a stable hosting infrastructure are essential steps that must be resolved before the system can be considered a full-scale, feature-complete tool.

5.5 Future Works

Moving into the FYP 2 phase next semester, the primary focus will shift from preliminary prototyping to full-scale system deployment and comprehensive evaluation. Building upon the infrastructure challenges identified in Chapter 4, the immediate future work involves finalizing the migration of the backend scripts to the Fly.io server to guarantee 24/7 uptime. Additionally, the integration of the Qwen/Ministral SLM will be optimized through refined prompt engineering to ensure the strict 30-word summarisation limits are consistently met.

Furthermore, to address the constraints identified within the current RAG setup, future development will focus on upgrading the system's question-answering architecture. Rather than relying on the current in-memory cosine similarity approach and basic string connection, the project will explore the integration of a dedicated, scalable external vector database. Alongside this infrastructure upgrade, more advanced text-chunking and semantic retrieval strategies will be investigated. This enhancement aims to safely expand the candidate pool and provide the language model

with deeper, more accurate context for answering complex user questions without compromising system speed.

Once the upgraded cloud and retrieval architectures are fully stabilized, the system will undergo its final evaluation phase. The survey sample size will be expanded from the current 86 pilot testers to the target goal of 150 respondents to achieve higher statistical reliability. Finally, the AI-generated summaries will be systematically evaluated using the established NLP metrics discussed in Chapter 2 specifically relevance, coherence, accurate consistency, and text compression to ensure the bot delivers accurate and high-quality local Sarawak news updates to the community.

REFERENCES

Alshamrani, A., & Bahattab, A. (2015). A comparison between three SDLC models waterfall model, spiral model, and Incremental/Iterative model. *International Journal of Computer Science Issues (IJCSI)*, *12*(1), 106–111.

Alaria, S. K., & Kaloriya, S. (2023). Comparative study of Java and Python: A review. *Industrial Engineering Journal*, *52*(4), 2698-2708.

Anjos, A. B. d. (2025). *Evaluating the Feasibility of Local LLMs in Privacy-Sensitive Environments: A Comparative Performance Framework* (Master's thesis). NOVA Information Management School, Universidade Nova de Lisboa.

Herasimenka, A., Bright, J., Knuutila, A., & Howard, P. N. (2023). Misinformation and professional news on largely unmoderated platforms: the case of telegram. *Journal of Information Technology and Politics*, *20*(2), 198–212. https://doi.org/10.1080/19331681.2022.2076272

Imroz, A., Rauf, M. A., & Phul, Z. (2023). Comparative case study: An evaluation of performance computation between SQL and NoSQL database. *Sindh Journal of Headways in Software Engineering*, *1*(2), 15-23.

Jerome, C., Ting, S. H., Hadzmy, A. J. A., & Raslie, H. (2023). Accessing News in the Digital Era: The Case of Sarawak, Malaysia. *5th Kuala Lumpur International Conference on Education, Economics and Technology (KLICEET2023)*.

Joshi, A. (2023). *The bias report: An automated news aggregator for political bias classification and news summarization* (Master's project). San Jose State University, SJSU ScholarWorks. https://doi.org/10.31979/etd.8rcp-3gaf

Kantar Media. (2016). *Brand and trust in a fragmented news environment: Qualitative research report*. Reuters Institute for the Study of Journalism.

Kümpel, A. S. (2021). Using Messaging Apps in Audience Research: An approach to study everyday information and news use practices. *Digital Journalism*, 10(1), 188–199. https://doi.org/10.1080/21670811.2020.1864219

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., & Rocktäschel, T. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Murugaiyan, D. (2012). *International Journal of Information Technology and Business Management WATEERFALLVs V-MODEL Vs AGILE: A COMPARATIVE STUDY ON SDLC*. *2*(1). www.jitbm.com

Othman, M. K., Norman Anuar, N., Barawi, M. H., Aman, S., Saili, J., & Abdullah Bandar, N. F. (2026). Social Media, Crowdsourcing and Everyday Life Information-Seeking: Cultural Heritage Preservation in Borneo Rural Communities. *Journal on Computing and Cultural Heritage*, *19*(1). https://doi.org/10.1145/3771927

Ren, J., Dong, H., Popovic, A., Sabnis, G., & Nickerson, J. (2024). Digital platforms in the news industry: how social media platforms impact traditional media news viewership. *European Journal of Information Systems*, *33*(1), 1–18. https://doi.org/10.1080/0960085X.2022.2103046

Røsok-Dahl, H., & Ihlebæk, K. A. (2024). Young People and News: A Systematic Literature Review. *Journalism Studies*, *25*(10), 1228–1248. https://doi.org/10.1080/1461670X.2024.2372425

Saravanos, A., & Curinga, M. X. (2023). Simulating the Software Development Lifecycle: The Waterfall Model. *Applied System Innovation*, *6*(6). https://doi.org/10.3390/asi6060108

Sulistiani, I. (2025). PROFESSIONAL COMMUNICATION IN THE DIGITAL AGE: BENEFITS AND CHALLENGES OF USING INSTANT MESSAGING APPLICATIONS IN THE WORKPLACE. In *International Journal of Society Reviews (INJOSER)* (Vol. 3, Issue 2).

Singh, S. (2025). Comparative analysis of Python's role in AI and machine learning. *International Journal of Research in Engineering, Science and Management*, *8*(2), 320-360.

Urnikienė, J., Steponavičienė, V., & Atanasov, S. (2026). Comparative read performance analysis of PostgreSQL and MongoDB in e-commerce: An empirical study of filtering and analytical queries. *MDPI*, 1-23.

Victor, S. A., Ibrahim, M. S., Yusuf, S., Mahmud, N., Bahari, K. A., Yoke Ling, L., & Abd Mubin, N. N. (2024). Social media addiction and depression among adolescents in two Malaysian states. *International Journal of Adolescence and Youth*, *29*(1). https://doi.org/10.1080/02673843.2023.2292055

Weng, Y., & Wu, J. (2025). Database management systems for artificial intelligence: Comparative analysis of PostgreSQL and MongoDB. *World Journal of Advanced Research and Reviews*, *25*(2), 2336-2342. https://doi.org/10.30574/wjarr.2025.25.2.0586

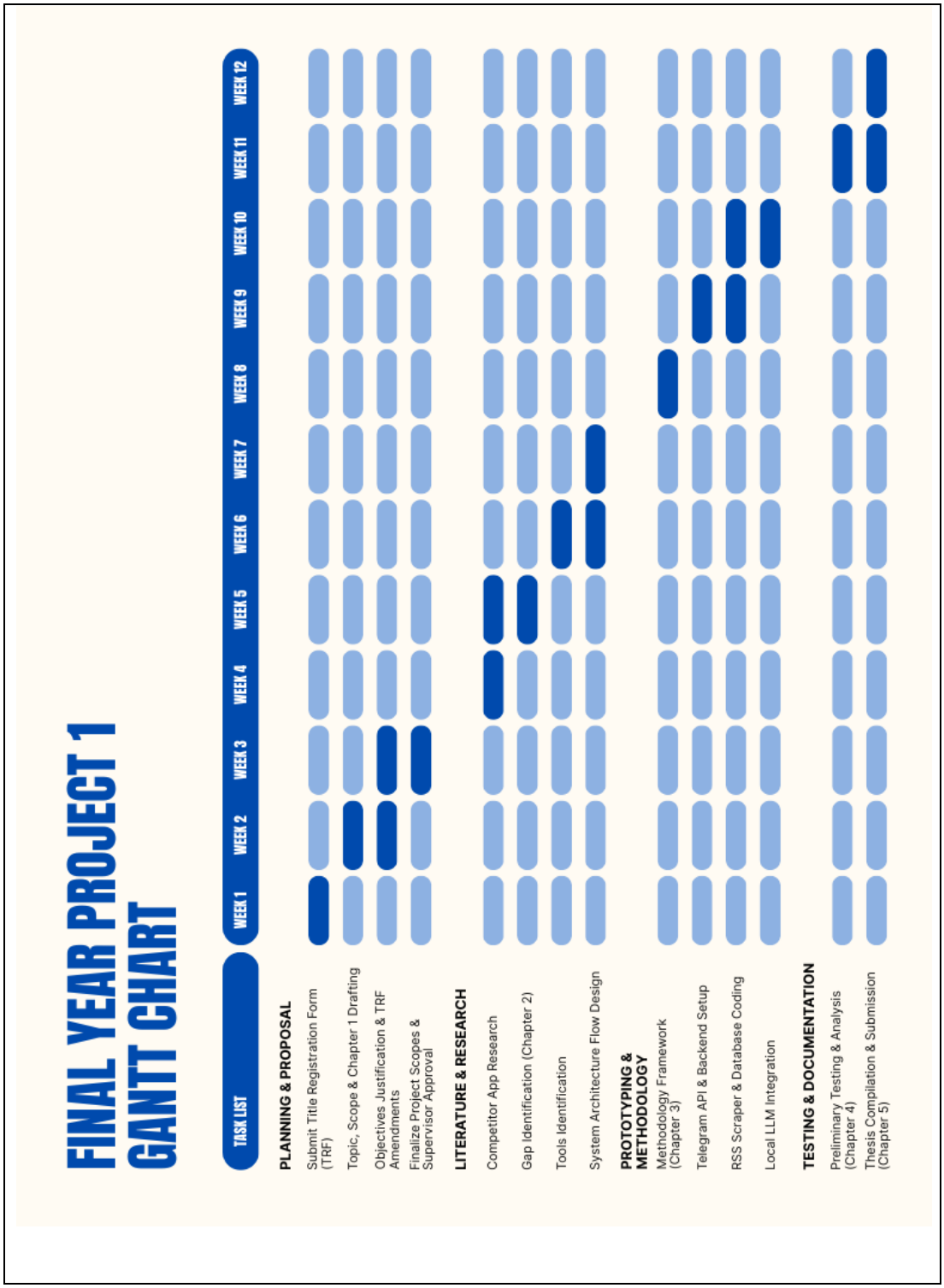
Xu, B., Chen, Y., Wen, Z., Liu, W., & He, B. (2024). *Evaluating Small Language Models for News Summarization: Implications and Factors Influencing Performance* (Vol. 1). Long Papers. https://arxiv.org/pdf/2502.00641

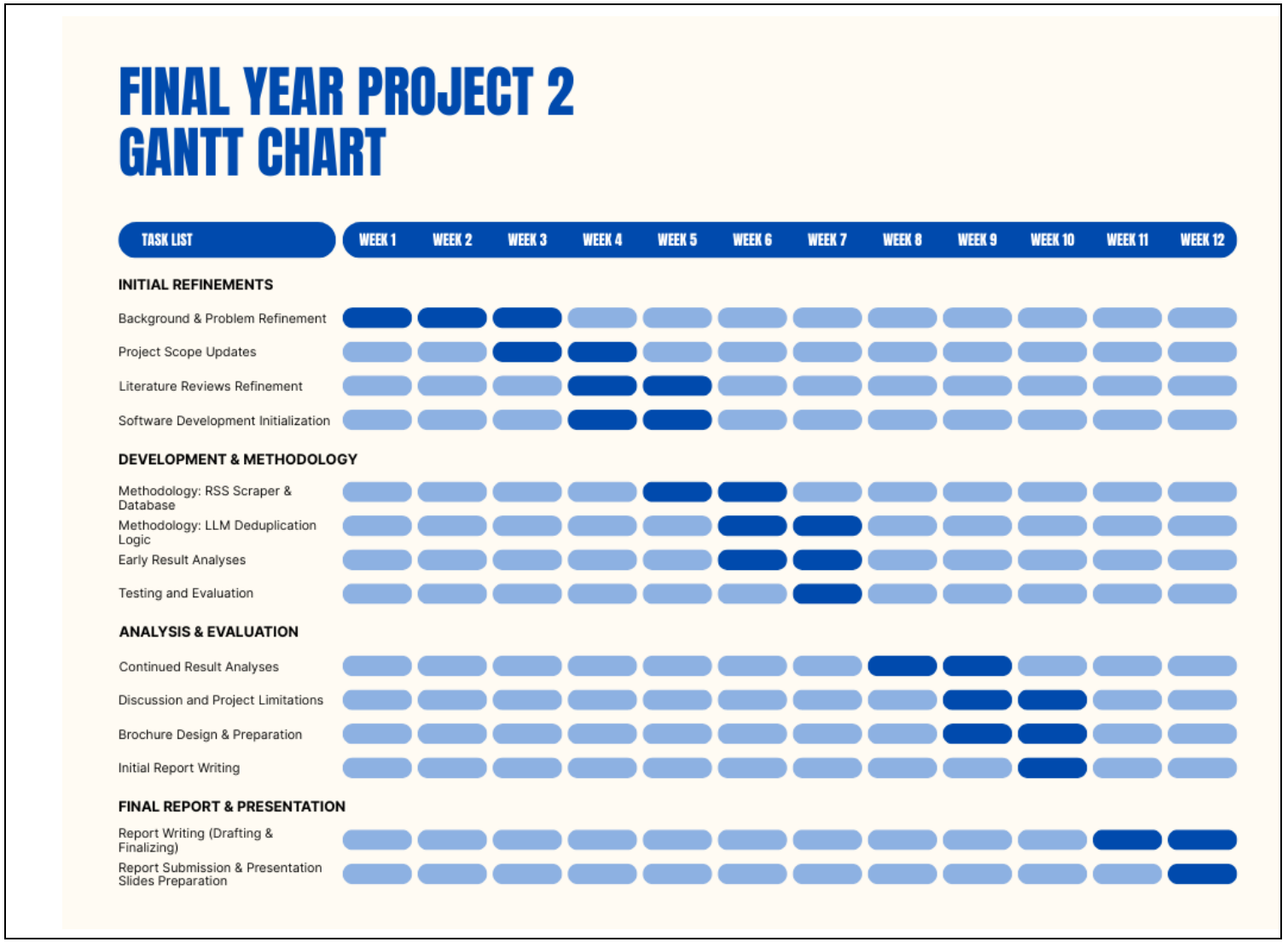
Yazdankhah, E. (2025). *Comparative analysis of backend-as-a-service solutions for React Native mobile applications: Firebase vs Supabase* (Bachelor's thesis). Haaga-Helia University of Applied Sciences.

Zhang, H., Yu, P. S., & Zhang, J. (2025). A Systematic Survey of Text Summarization: From Statistical Methods to Large Language Models. *ACM Computing Surveys*, *57*(11). https://doi.org/10.1145/3731445

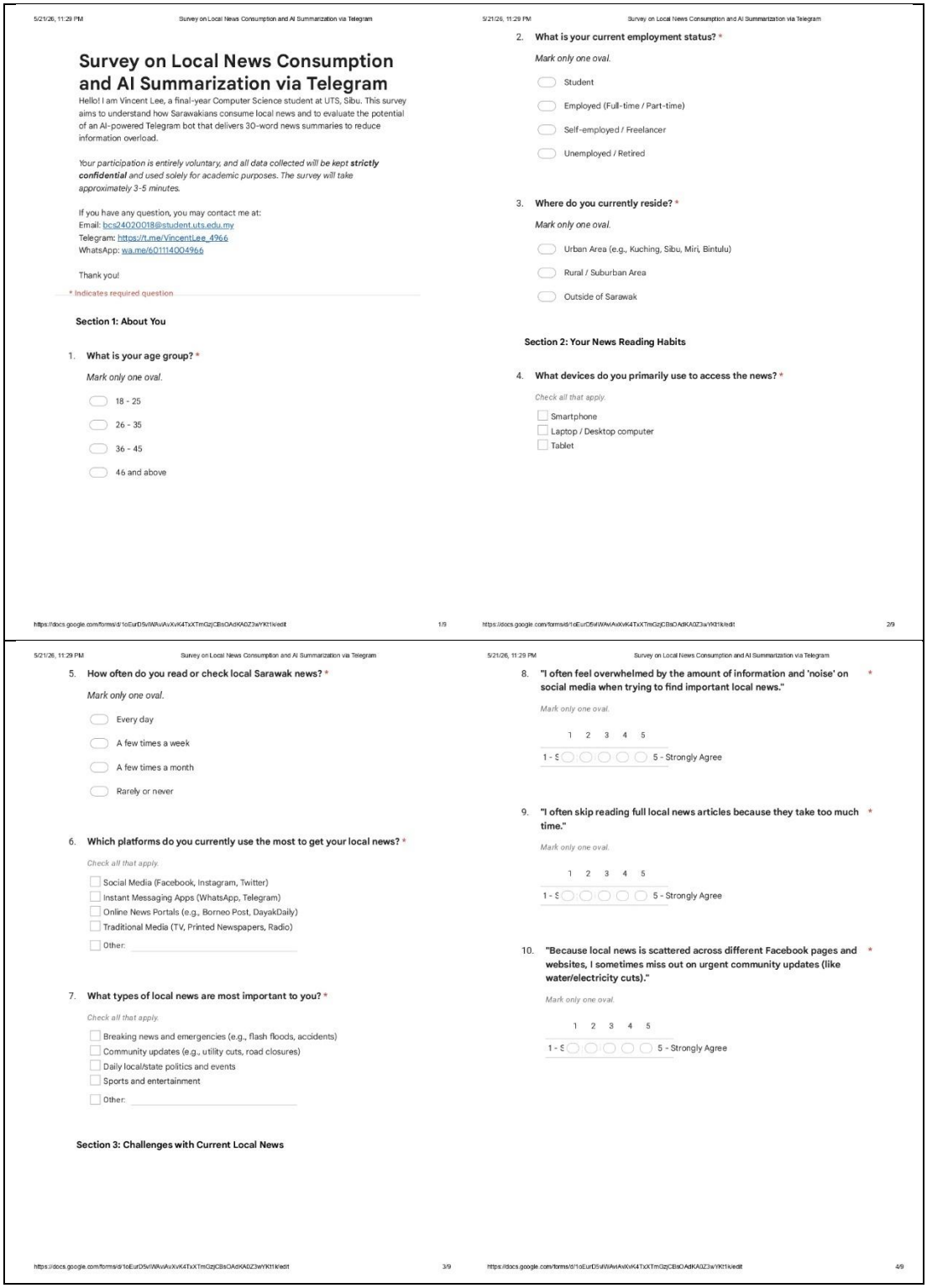
Zhang, T., Ladhak, F., Durmus, E., Liang, P., McKeown, K., & Hashimoto, T. B. (2024). Benchmarking large language models for news summarization. *Transactions of the Association for Computational Linguistics*, *12*, 39–57. https://doi.org/10.1162/tacl_a_00632

APPENDIX A: FYP1 & FYP2 GANTT CHART




FINAL YEAR PROJECT 2
GANTT CHART
TASK LIST
WEEK 1
WEEK 2
WEEK 3
WEEK 4
WEEK 5
WEEK 6
WEEK 7
WEEK 8
WEEK 9
WEEK 10
WEEK 11
WEEK 12
INITIAL REFINEMENTS
Background & Problem Refinement
Project Scope Updates
Literature Reviews Refinement
Software Development Initialization
DEVELOPMENT & METHODOLOGY
Methodology: RSS Scraper & Database
Methodology: LLM Deduplication Logic
Early Result Analyses
Testing and Evaluation
ANALYSIS & EVALUATION
Continued Result Analyses
Discussion and Project Limitations
Brochure Design & Preparation
Initial Report Writing
FINAL REPORT & PRESENTATION
Report Writing (Drafting & Finalizing)
Report Submission & Presentation Slides Preparation

APPENDIX B: QUESTIONNAIRE



5/21/26, 11:29 PM Survey on Local News Consumption and AI Summarization via Telegram

Survey on Local News Consumption and AI Summarization via Telegram

Hello! I am Vincent Lee, a final-year Computer Science student at UTS, Sibu. This survey aims to understand how Sarawakians consume local news and to evaluate the potential of an AI-powered Telegram bot that delivers 30-word news summaries to reduce information overload.

*Your participation is entirely voluntary, and all data collected will be kept **strictly confidential** and used solely for academic purposes. The survey will take approximately 3-5 minutes.*

If you have any question, you may contact me at:
Email: bcs24020018@student.uts.edu.my
Telegram: https://t.me/VincentLee_4966
WhatsApp: wa.me/601114004966

Thank you!

* Indicates required question

Section 1: About You

1. **What is your age group?** *

Mark only one oval.

- 18 - 25
- 26 - 35
- 36 - 45
- 46 and above

https://docs.google.com/forms/d/1oEurD5vWAviAvXvK4TxXTmGzjCBsOAdKA0Z3wYKt1k/edit 1/9

5/21/26, 11:29 PM Survey on Local News Consumption and AI Summarization via Telegram

2. **What is your current employment status?** *

Mark only one oval.

- Student
- Employed (Full-time / Part-time)
- Self-employed / Freelancer
- Unemployed / Retired

3. **Where do you currently reside?** *

Mark only one oval.

- Urban Area (e.g., Kuching, Sibu, Miri, Bintulu)
- Rural / Suburban Area
- Outside of Sarawak

Section 2: Your News Reading Habits

4. **What devices do you primarily use to access the news?** *

Check all that apply.

- [ ] Smartphone
- [ ] Laptop / Desktop computer
- [ ] Tablet

https://docs.google.com/forms/d/1oEurD5vWAviAvXvK4TxXTmGzjCBsOAdKA0Z3wYKt1k/edit 2/9

5/21/26, 11:29 PM Survey on Local News Consumption and AI Summarization via Telegram

5. **How often do you read or check local Sarawak news?** *

Mark only one oval.

- Every day
- A few times a week
- A few times a month
- Rarely or never

6. **Which platforms do you currently use the most to get your local news?** *

Check all that apply.

- [ ] Social Media (Facebook, Instagram, Twitter)
- [ ] Instant Messaging Apps (WhatsApp, Telegram)
- [ ] Online News Portals (e.g., Borneo Post, DayakDaily)
- [ ] Traditional Media (TV, Printed Newspapers, Radio)
- [ ] Other: _____

7. **What types of local news are most important to you?** *

Check all that apply.

- [ ] Breaking news and emergencies (e.g., flash floods, accidents)
- [ ] Community updates (e.g., utility cuts, road closures)
- [ ] Daily local/state politics and events
- [ ] Sports and entertainment
- [ ] Other: _____

Section 3: Challenges with Current Local News

https://docs.google.com/forms/d/1oEurD5vWAviAvXvK4TxXTmGzjCBsOAdKA0Z3wYKt1k/edit 3/9

5/21/26, 11:29 PM Survey on Local News Consumption and AI Summarization via Telegram

8. **"I often feel overwhelmed by the amount of information and 'noise' on social media when trying to find important local news."** *

Mark only one oval.

1 2 3 4 5

1 - S ○ ○ ○ ○ ○ 5 - Strongly Agree

9. **"I often skip reading full local news articles because they take too much time."** *

Mark only one oval.

1 2 3 4 5

1 - S ○ ○ ○ ○ ○ 5 - Strongly Agree

10. **"Because local news is scattered across different Facebook pages and websites, I sometimes miss out on urgent community updates (like water/electricity cuts)."** *

Mark only one oval.

1 2 3 4 5

1 - S ○ ○ ○ ○ ○ 5 - Strongly Agree

https://docs.google.com/forms/d/1oEurD5vWAviAvXvK4TxXTmGzjCBsOAdKA0Z3wYKt1k/edit 4/9

Section 4: Live Bot Testing & Feedback

To solve these problems, I am developing an AI Local News Summarization Bot on Telegram. This bot automatically gathers local Sarawak news, uses a local AI model to summarize lengthy articles into short 30-word briefs, and sends them directly to your Telegram app as push notifications.

Time to test the prototype! 🚀

Please take 2 minutes to try the actual AI News Bot on Telegram before answering the questions below.

Step 1: Click this link:

@apanews_bot (or scan the QR code below).

Step 2: Click **Start** in Telegram and follow the instruction, clicking "Finish and get the latest news" at the end.

Step 3: Come back to this form to share your experience!

Try @apanews_bot on Telegram now by click the link or scan the QR code below.



11. Were you able to successfully access and interact with the Telegram bot? *

Mark only one oval.

- Yes
- No (If no, please let me know what went wrong in the final feedback box)

12. "The AI-generated news summaries were easy to read and gave me the core information quickly." *

Mark only one oval.

1 2 3 4 5

1 - S ◯ ◯ ◯ ◯ ◯ 5 - Strongly Agree

13. "Navigating the bot using the menu buttons (e.g., Settings, Latest, Help) was simple and straightforward." *

Mark only one oval.

1 2 3 4 5

1 - S ◯ ◯ ◯ ◯ ◯ 5 - Strongly Agree

14. "I found the "Settings" feature useful for customizing the news to my specific location and interests." *

Mark only one oval.

1 2 3 4 5

1 - S ◯ ◯ ◯ ◯ ◯ 5 - Strongly Agree

15. "Overall, I would use this bot in my daily life to stay updated on local Sarawak news." *

Mark only one oval.

1 2 3 4 5

1 - S ◯ ◯ ◯ ◯ ◯ 5 - Strongly Agree

16. Do you have any suggestions to improve the bot (More languages, etc), or did you encounter any errors?

Thank You!

Thank you for taking the time to test my Final Year Project! If you are open to being contacted for a short follow-up regarding your experience, please leave your Telegram handle or email below.

If you have any question or feedback, you may contact me at:
Email: bcs24020018@student.uts.edu.my
Telegram: https://t.me/VincentLee_4966
WhatsApp: wa.me/601114004966

17. **Please leave your Telegram Handle (e.g., @yourname), email address or Phone/WhatsApp number.**

This content is neither created nor endorsed by Google.

Google Forms

APPENDIX C: GOOGLE FORM RESPONSES

